

Can you recognise the music playing?

6COSC020W: APPLIED AI

WEEK 8: Neural Networks (NNs)

Dr. Ester Bonmati

(e.bonmaticoll@westminster.ac.uk)

While you wait: **pollev.com/esterbonmati**
What is a neural network?



Before we start...

1

Poll everywhere

Polls at pollev.com/esterbonmati. Answers appear only after you vote.

2

Pen and paper

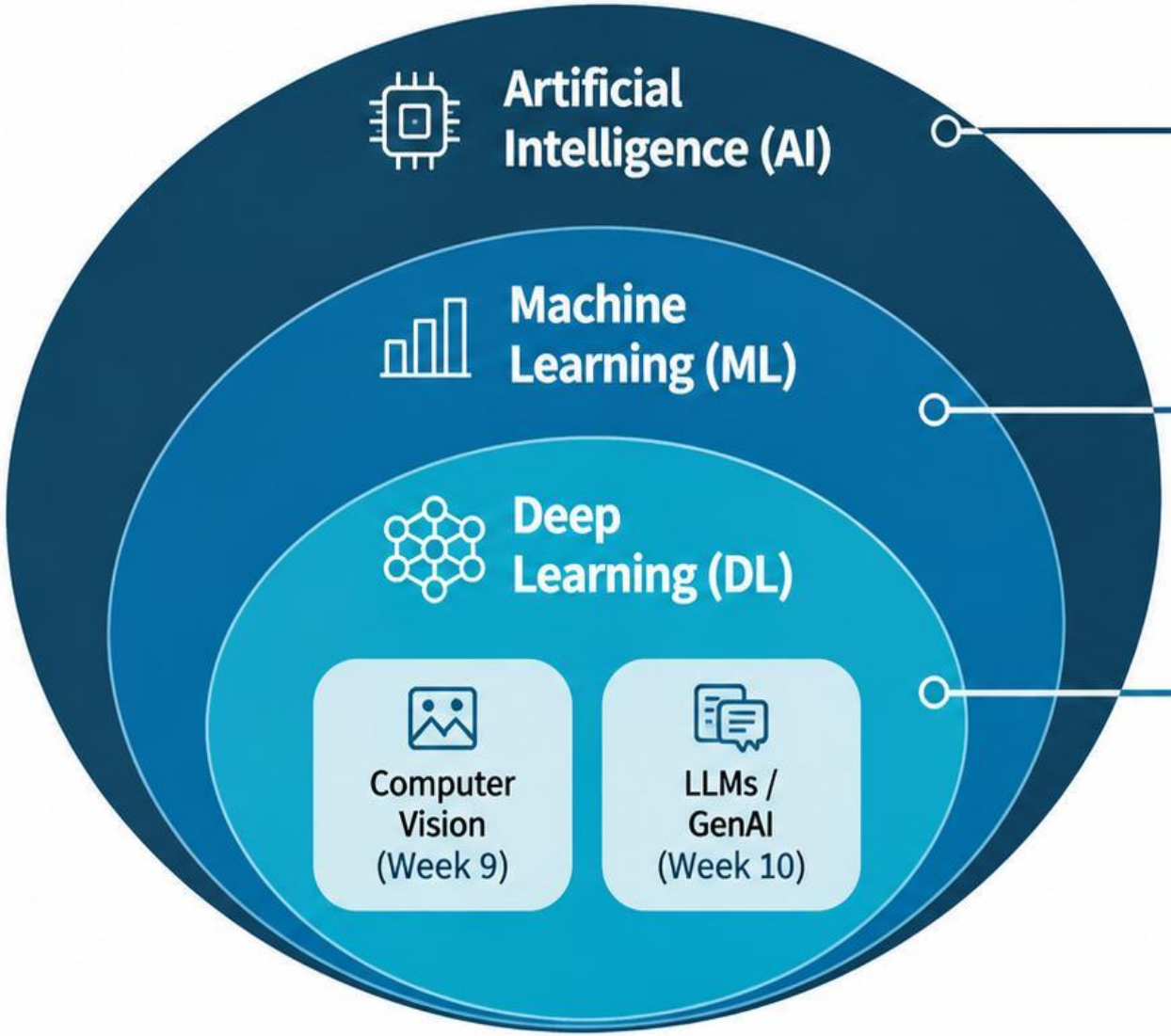
Some questions need a calculation. Use a notebook or your phone's notes.

3

Engage

Participate in the lecture by responding to questions, asking questions ...

Today: Neural Networks



Artificial Intelligence (AI)

The broad field of building systems that exhibit intelligent behaviour.

Weeks 1–5, 11

Machine Learning (ML)

A subfield of AI that learns patterns from data instead of being explicitly programmed.

Week 7

Deep Learning (DL) / Neural Networks

A subfield of ML that uses multi-layer neural networks to learn complex representations from large amounts of data.

Week 8

Learning outcomes

1. **Calculate** the output of a perceptron: weighted sum, bias, activation
2. **Explain** how a perceptron learns from its mistakes
3. **Explain** why one neuron is not enough, and what hidden layers add
4. **Understand** loss, gradients and backpropagation.
5. **Diagnose** overfitting and a bad learning rate from learning curves
6. **Evaluate** a model: validation, hyperparameters, accuracy and class imbalance

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Can we teach a computer to recognise spam?

A

Subject: You have WON!!!

Dear winner, claim your FREE holiday today. FREE flights, FREE hotel.

Click here: bit.ly/win-now

B

Subject: Assignment 2 feedback

Hi Sam, your feedback is ready: moodle.example.ac.uk/feedback

See you on Thursday. Dr Lee

Which of these two emails is spam? Why?

From cues to numbers

Email	Exclamation marks (x_1)	Links (x_2)	"free" count (x_3)
A	3	1	3
B	0	1	0

A → (3, 1, 3)
B → (0, 1, 0)

Each email becomes a list of numbers. *What else would you measure?*

Nobody writes the rules

Write the rules yourself

```
if exclamation marks > 2:
    spam
if "free" in text:
    spam
…and a new rule for every trick
```

Learn them from examples

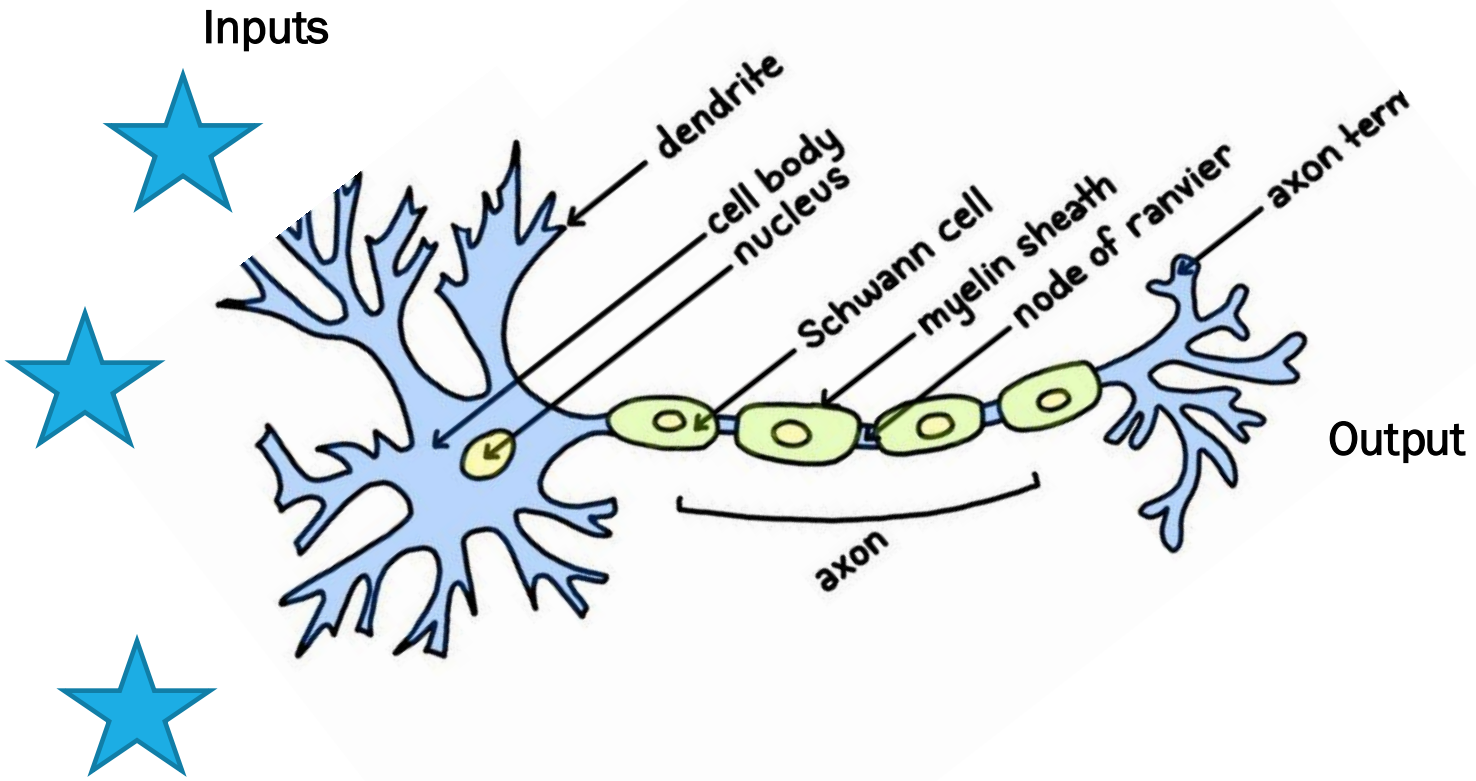
Labelled emails (spam / not spam) go in. A model with adjustable numbers comes out. Training finds the numbers.

How can a computer learn to make decisions when nobody explicitly tells it the rules?



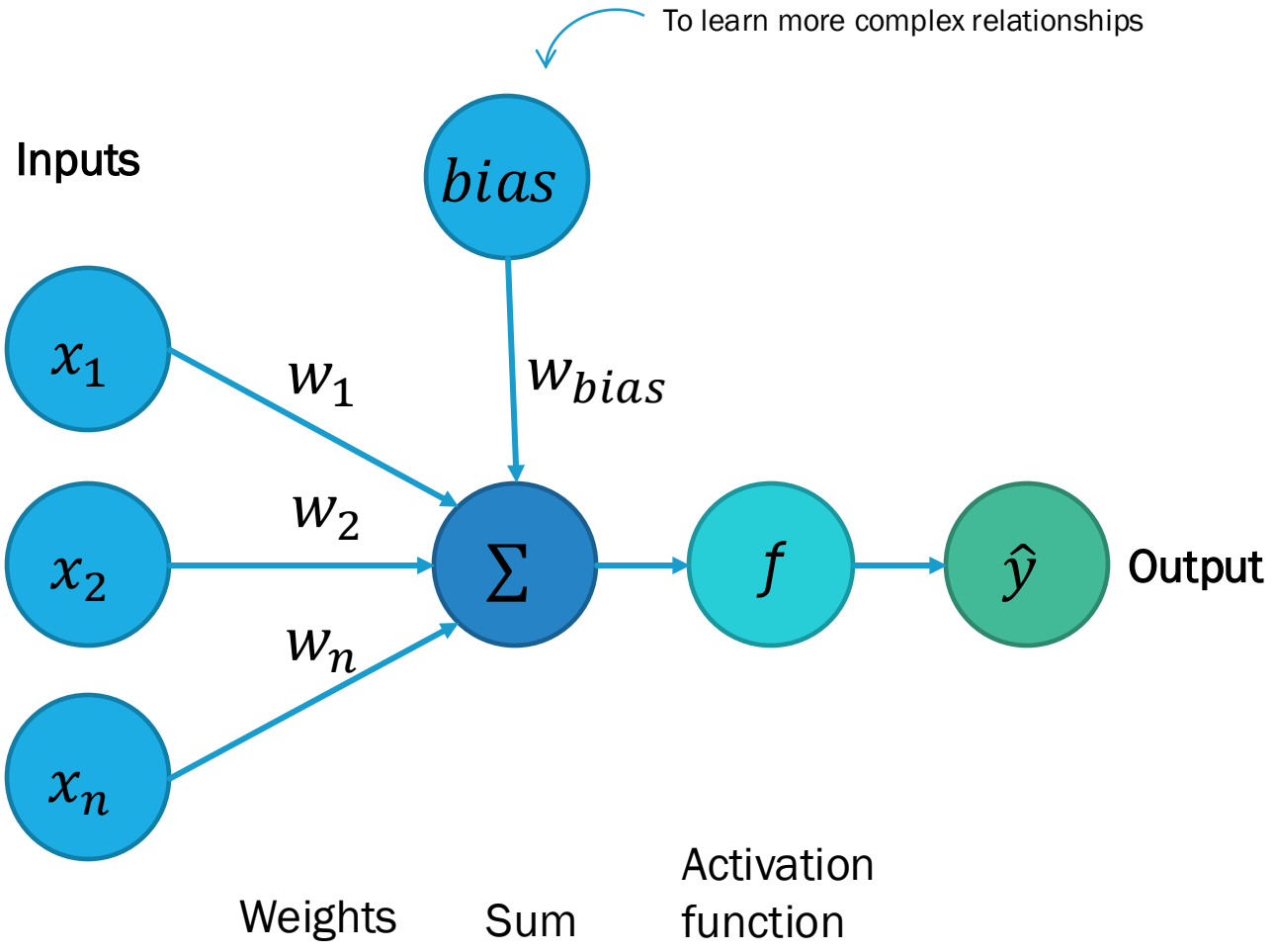
Neurons

Neuron



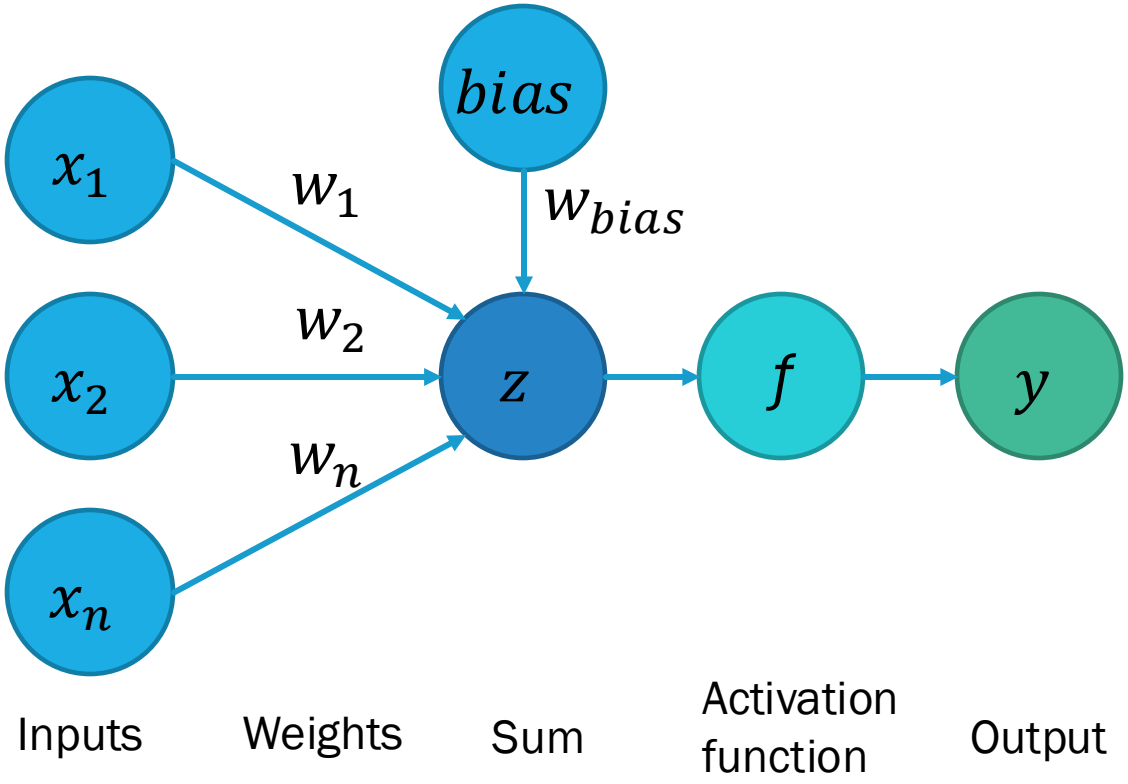
The human brain has roughly 86 billion neurons

Perceptron


To learn more complex relationships
Inputs
bias
x_1
x_2
x_n
w_1
w_2
w_n
w_{bias}
$\sum$
f
$\hat{y}$
Output
Weights
Sum
Activation function

The perceptron

- Building block of neural networks (single neuron)
- Mimics the neuron in the human brain
- **Forward propagation (input → neuron → output)**



Output
Bias
Inputs

$$y = f\left(bias\ w_{bias} + \sum_{i=1}^{n} x_i\ w_i\right)$$

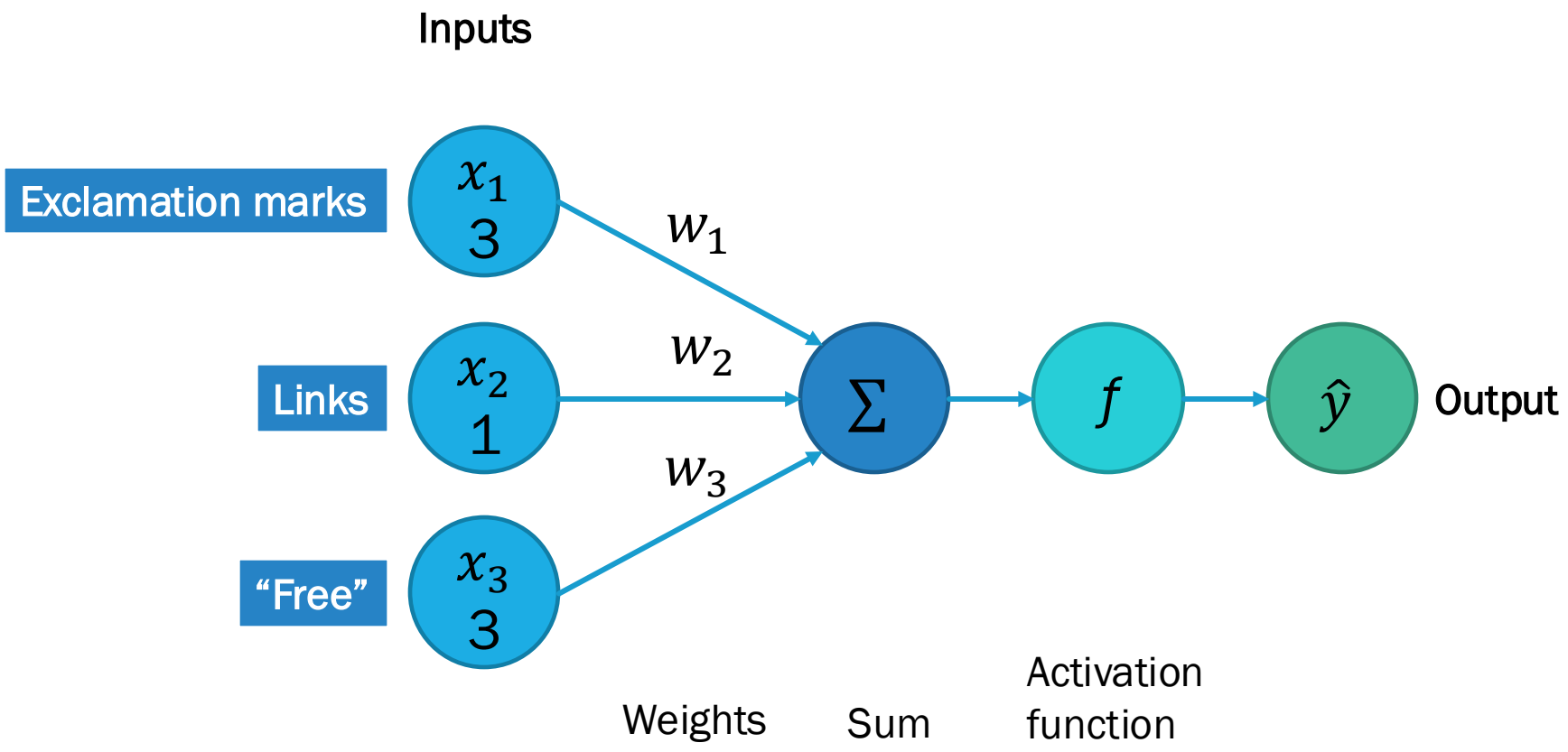
Activation function
Sum
Weights

$$z = x_1\ w_1 + x_2\ w_2 + \ldots + x_n\ w_n + bias\ w_{bias}$$

$$y = f(z)$$

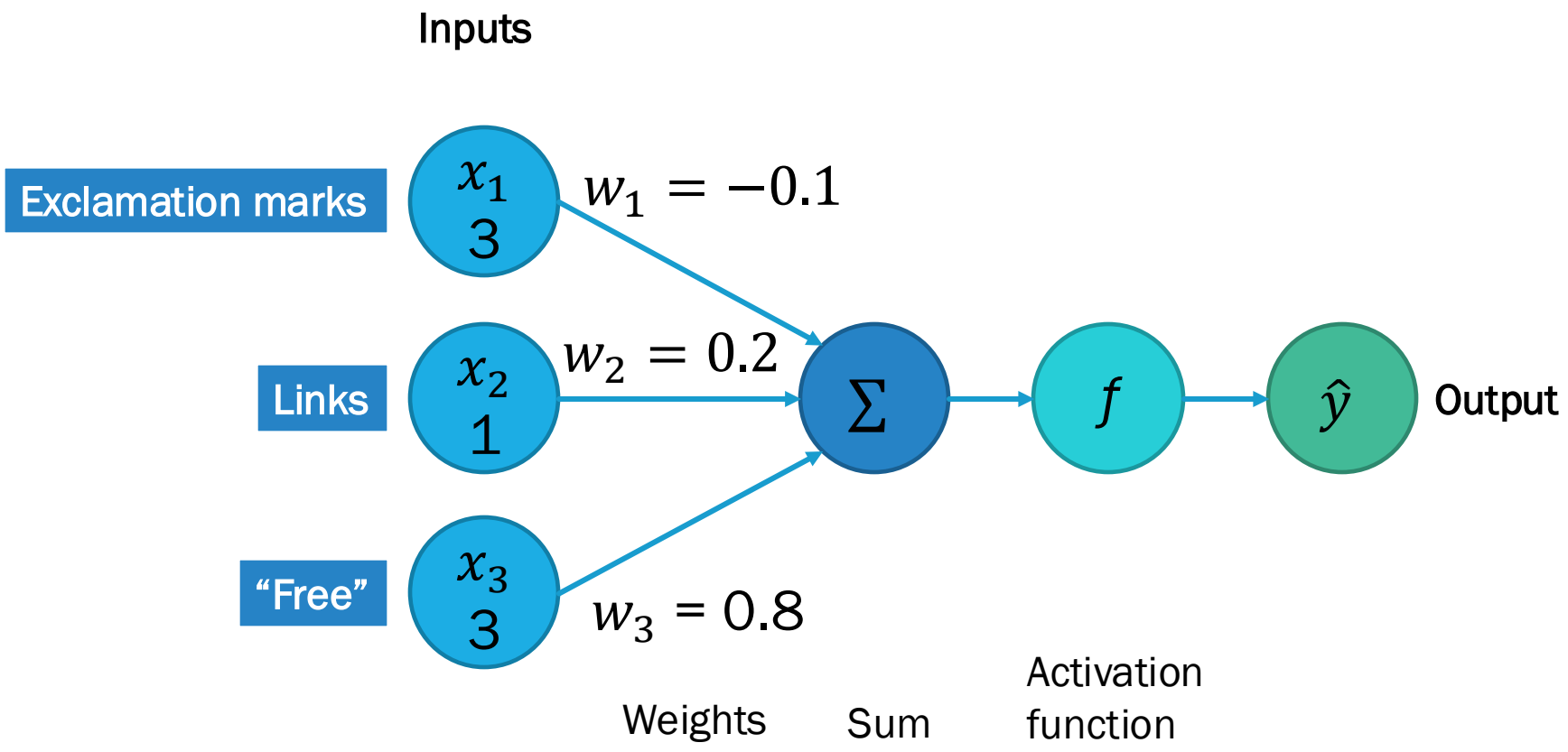
Example: $y = \begin{cases} 0\ if\ z \leq 0 \\ 1\ if\ z > 0 \end{cases}$

A perceptron turns features into a decision



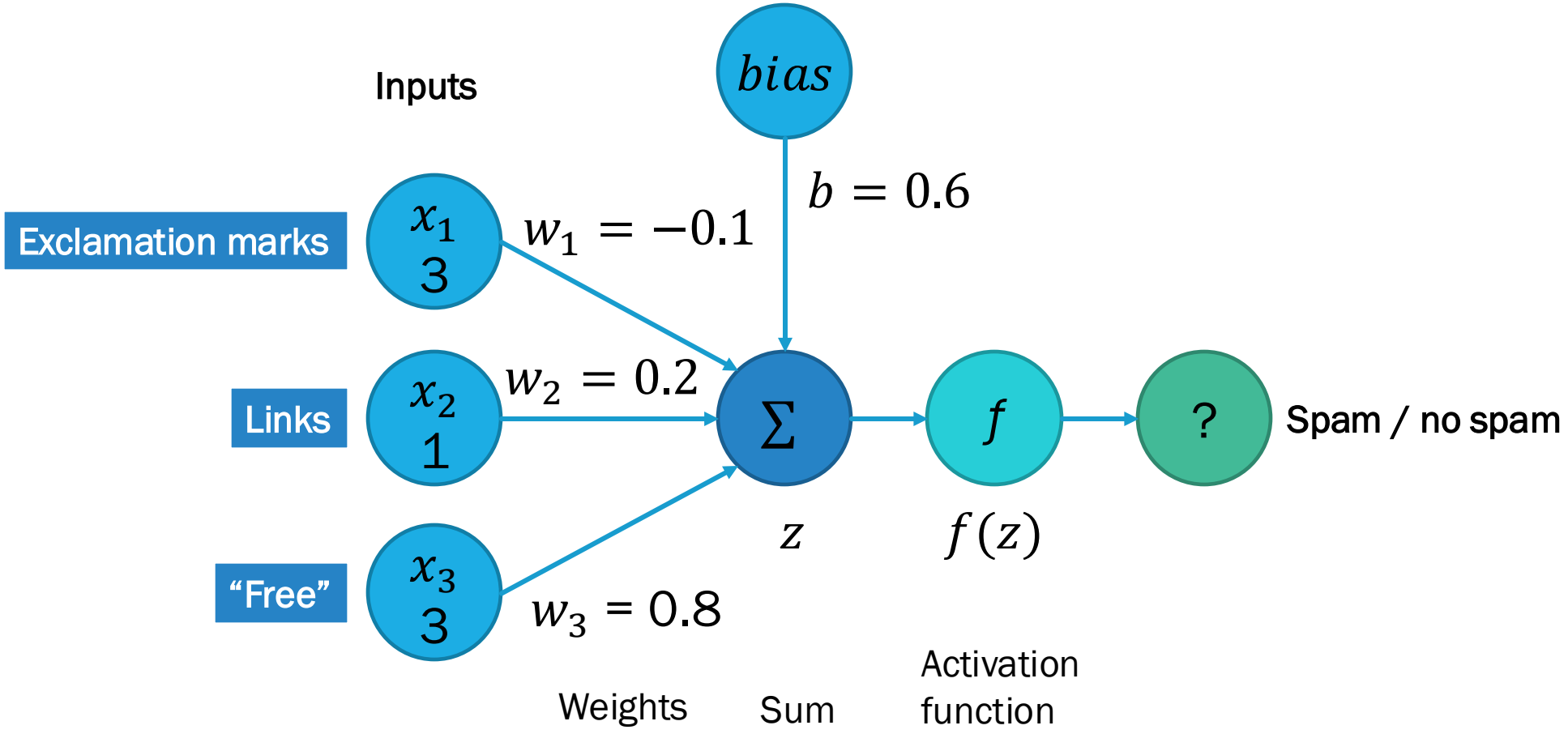
Question: How much should each input matter?

A perceptron turns features into a decision



Weights say how much each input matters. Random initially (not trained).

A perceptron turns features into a decision



Bias shifts the threshold. The activation function turns z into a 'yes' or 'no'

Forward propagation

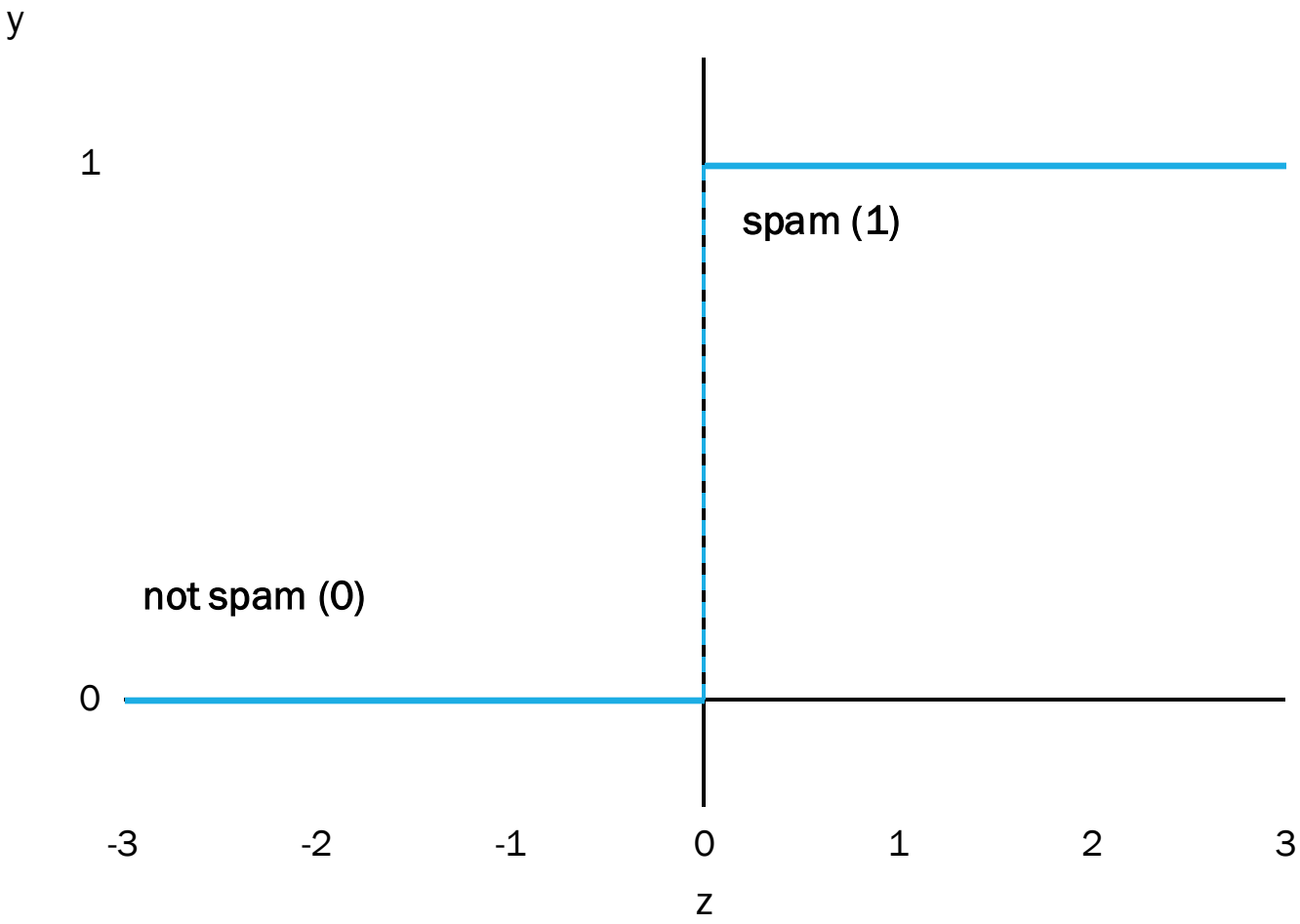
$$z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

$$y = f(z)$$

Activation function:
Turns z into a decision

$$y = \begin{cases} 1 \; if \; z > 0 \\ 0 \; if \; z \leq 0 \end{cases}$$

The activation: z becomes a decision



y = 1 if z > 0 (spam)
y = 0 otherwise (no spam)

How good is our ‘random’ perceptron?

Our data

A

Subject: You have WON!!!

Dear winner, claim your FREE holiday today. FREE flights, FREE hotel.

Click here: bit.ly/win-now

B

Subject: Assignment 2 feedback

Hi Sam, your feedback is ready: moodle.example.ac.uk/feedback

See you on Thursday. Dr Lee

Email	Exclamation marks (x_1)	Links (x_2)	"free" count (x_3)	Label
A	3	1	3	Spam
B	0	1	0	Not spam

Worked example: Email A

Feature	Value x	Weight w	x × w
Exclamation marks	3	−0.1	−0.3
Links	1	0.2	0.2
"free" count	3	0.8	2.4
Bias	1	0.6	0.6

z = -0.3 + 0.2+ 2.4 + 0.6

z = 2.9

2.9 > 0 → y = 1

spam ✓

y = 1 if z > 0 (spam)

y = 0 otherwise (no spam)

Your turn: Email B

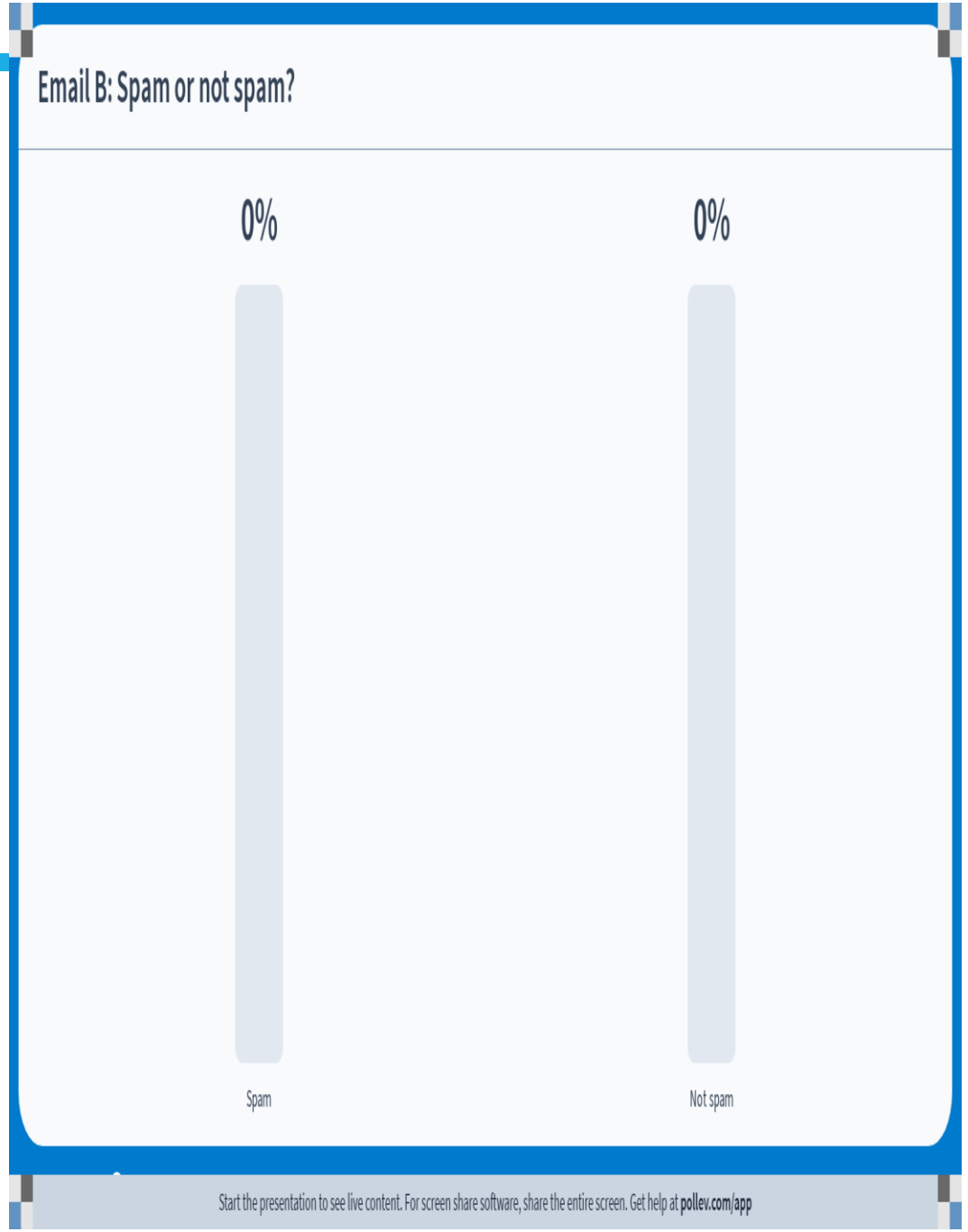
Feature	Value x	Weight w	x × w
Exclamation marks	0	−0.1	?
Links	1	0.2	?
"free" count	0	0.8	?
Bias	1	0.6	?

y = 1 if z > 0 (spam)
y = 0 otherwise (no spam)

1) Calculate z
2) Calculate y

Spam or not spam?

Compare answers with the person sitting next to you



Your turn: Email B

Feature	Value x	Weight w	x × w
Exclamation marks	0	−0.1	0
Links	1	0.2	0.2
"free" count	0	0.8	0
Bias	1	0.6	0.6

$z = 0.8$

$0.8 > 0 \rightarrow$ spam ✗

But B is not spam. The perceptron is wrong.

What can we do to make our perceptron better (more accurate)?



Update weights or bias

What happens if we update the weights?

- We change the **weight w_3 from 0.8 to 1.5 and lower the bias to -0.5**. What is the result now?

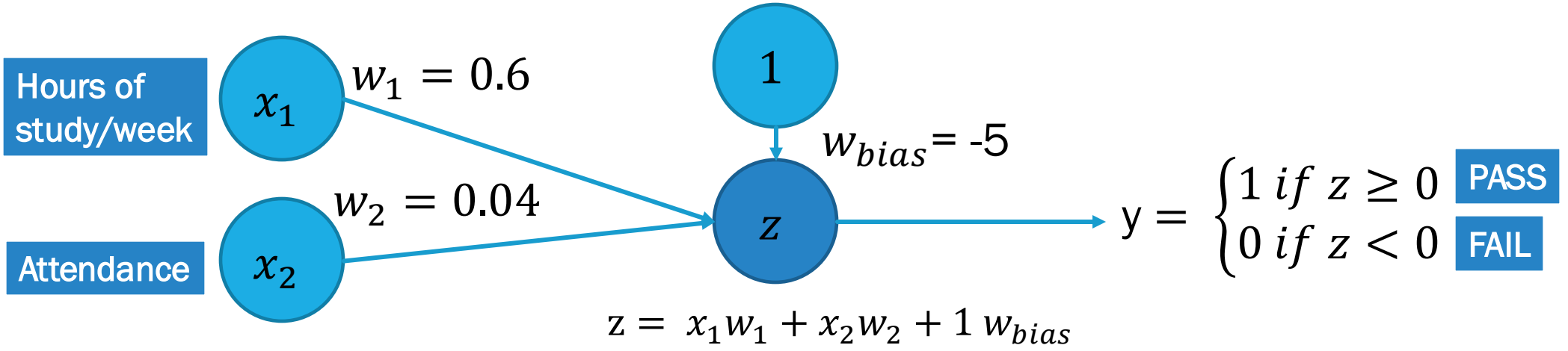
Email A	Value x	Weight w	x × w
Exclamation marks	3	−0.1	−0.3
Links	1	0.2	0.2
"free" count	3	**1.5**	**4.5**
Bias	1	**-0.5**	**-0.5**
z			**3.9**

Spam ✓

Email B	Value x	Weight w	x × w
Exclamation marks	0	−0.1	0
Links	1	0.2	0.2
"free" count	0	**1.5**	**0**
Bias	1	**-0.5**	**-0.5**
z			**-0.3**

No Spam ✓

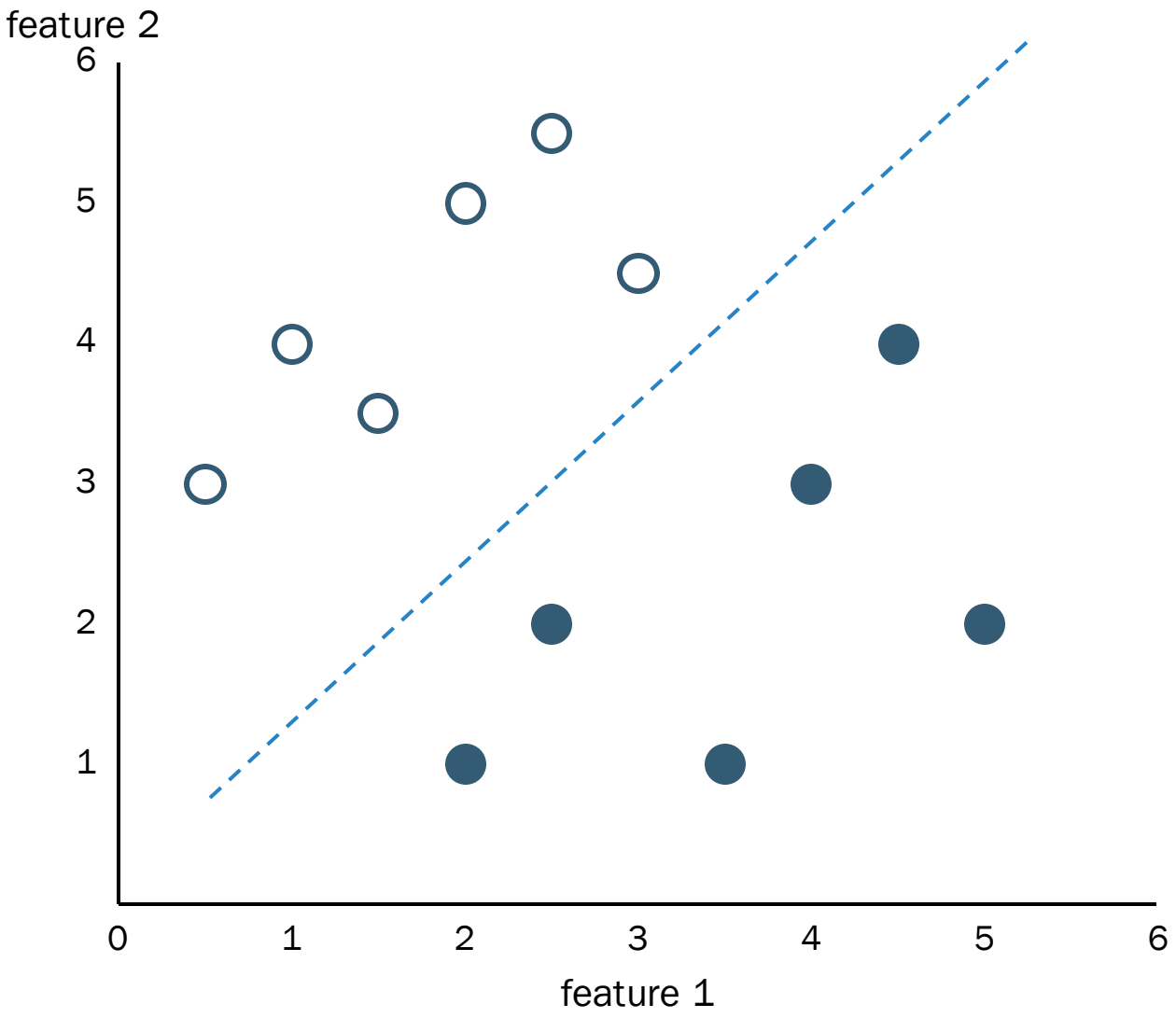
Perceptron: Another example



Hours Studied (x_1)	Attendance (x_2)	Weighted Sum	Output	Prediction
2	50	0.6x2 + 0.04x50 - 5 = -1.8	0	Fail
8	90	0.6x8 + 0.04x90 - 5 = 3.4	1	Pass
3	70	0.6x3 + 0.04x70 - 5 = -0.4	0	Fail
6	95	0.6x6 + 0.04x95 - 5 = 2.4	1	Pass

More data comes in….

The training set



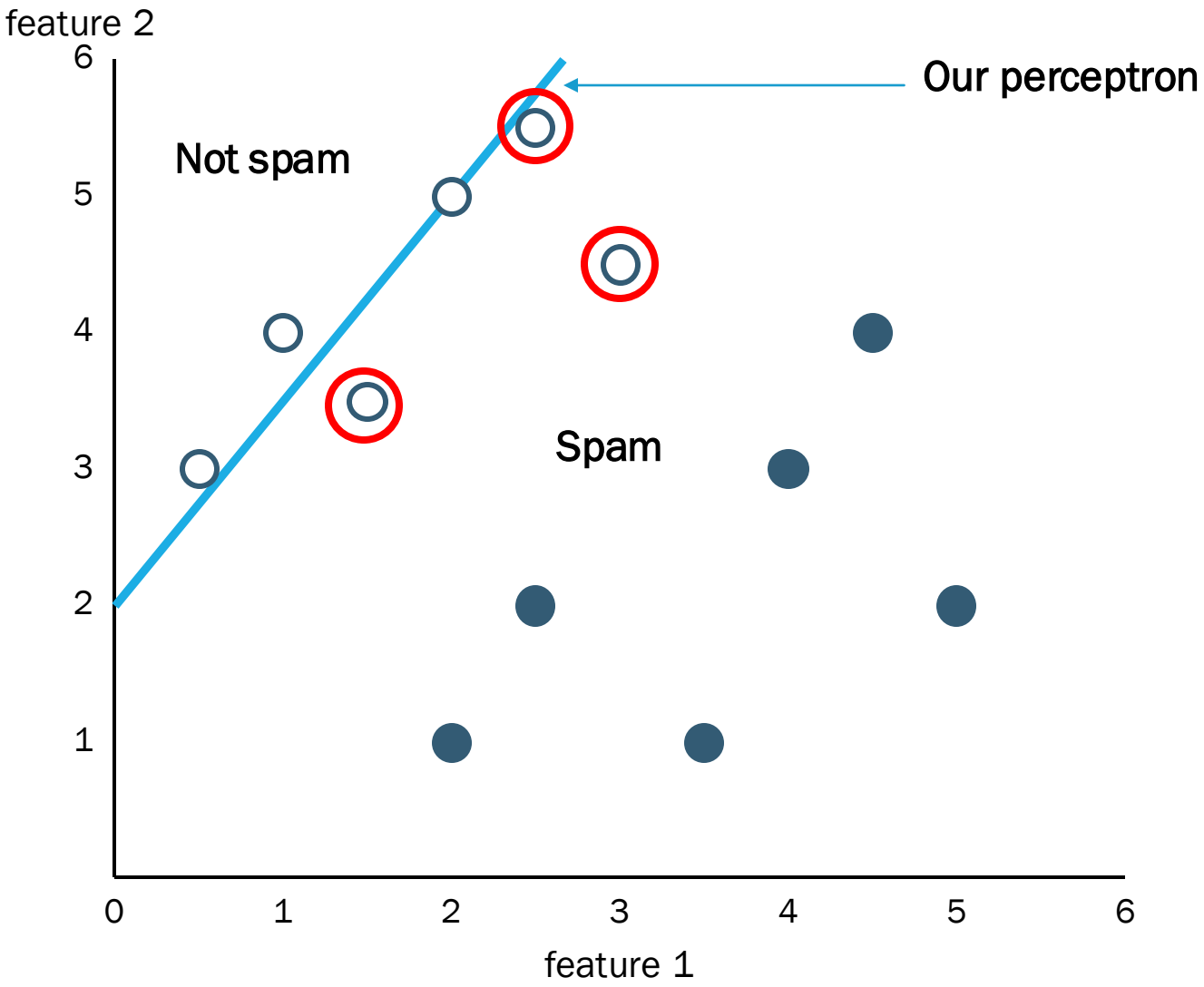
Each email is a point

Two features, two classes. A perceptron draws one straight line.

Can one straight line separate the filled points from the hollow ones?

spam (1) not spam (0)

Random weights: how many are right?



$w_1 = 0.75 \quad w_2 = -0.5 \quad b = 1$

line: $0.75x_1 - 0.5x_2 + 1 = 0$

How many of the 12 points are classified correctly?

Which way should we move the line?

spam (1) not spam (0)

When we have more data, manually changing the weights is no longer feasible.

How can we update the weights automatically to 'learn' to classify emails?

What is learning?

- Changing the weights and bias so that the prediction y matches the label t.

Training data (sample)

Email	x_1	x_2	x_3	Label t	Prediction y
A	3	1	3	1 Spam	1 ✓
B	0	1	0	0 Not spam	1 ✗

When $y \neq t$, we have an error

When the neuron makes a mistake, each weight is updated by the **learning rate** times the error times its input.

Parameter that controls how much the weights change during training

The perceptron learning rule: move z in the right direction

$$\text{new_w} \leftarrow \text{w} + \eta\,(t - y)\,\mathbf{x} \qquad \eta = \text{learning rate}$$

$$\text{new_b} \leftarrow \text{b} + \eta\,(t - y)$$

- If the neuron predicts spam (y = 1) but the label is not spam (t = 0), then t − y = −1: we want z to go DOWN, so we subtract the input from the weights.
- If it predicts not-spam but the label is spam, then t − y = +1: add the input.
- If it is right, t − y = 0 and nothing changes.

Fix Email B with the rule

	w_1	w_2	w_3	b	z for B	B is...
start	−0.1	0.2	0.8	0.6	0.8	spam ✗
update 1	−0.1	−0.05	0.8	0.35	0.30	spam ✗
update 2	−0.1	−0.30	0.8	0.10	−0.20	not spam ✓

Email B: x = (0, 1, 0), label t = 0 (not spam). $\eta = 0.25$. $w \leftarrow w + \eta (t - y) x$, $b \leftarrow b + \eta (t - y)$

Email A (spam): z = 1.9, still spam ✓

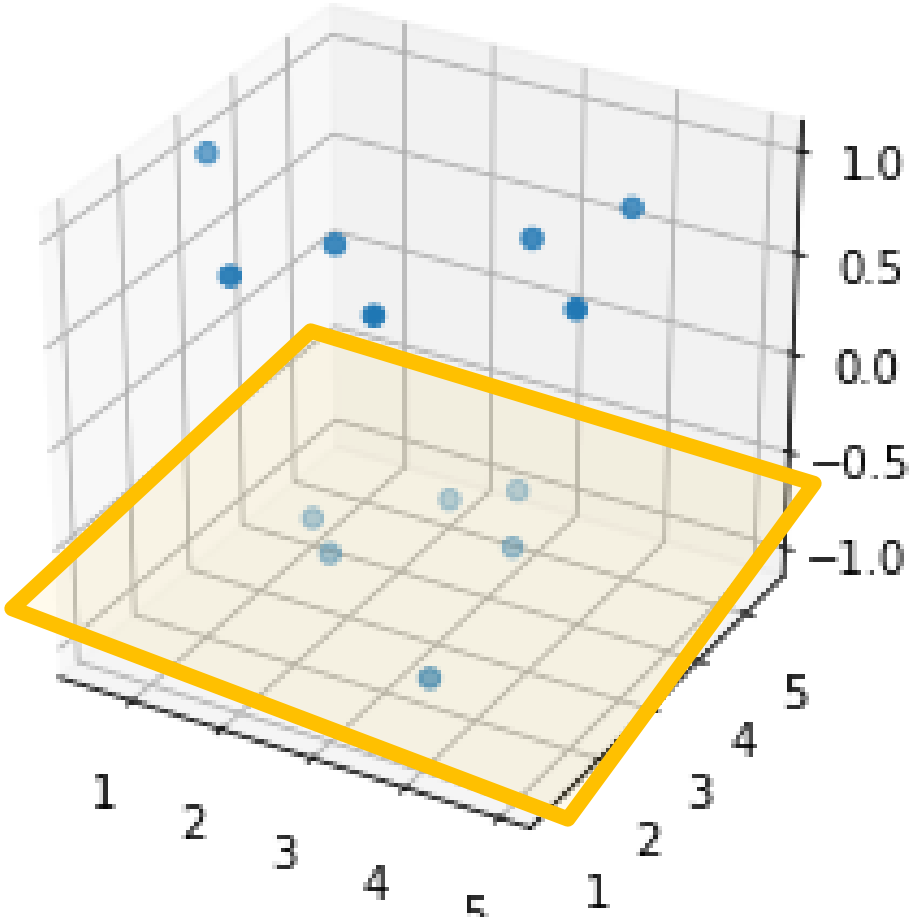
Email D (spam; 1 !, 2 links, no "free"): z 0.9 → −0.6, now missed ✗

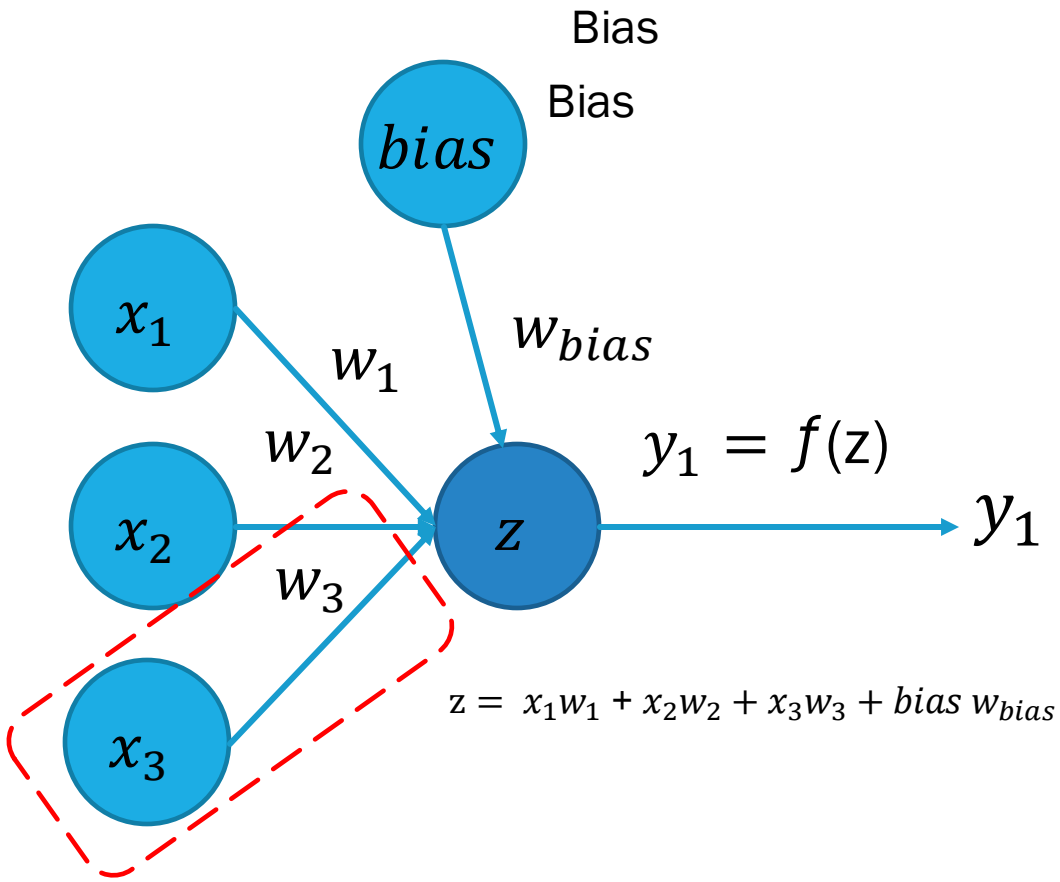
One fix can break another. Hence: loop over many examples.

Keep going until nothing is wrong. Iterate through the whole dataset.

Neural networks: another example

$$y = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

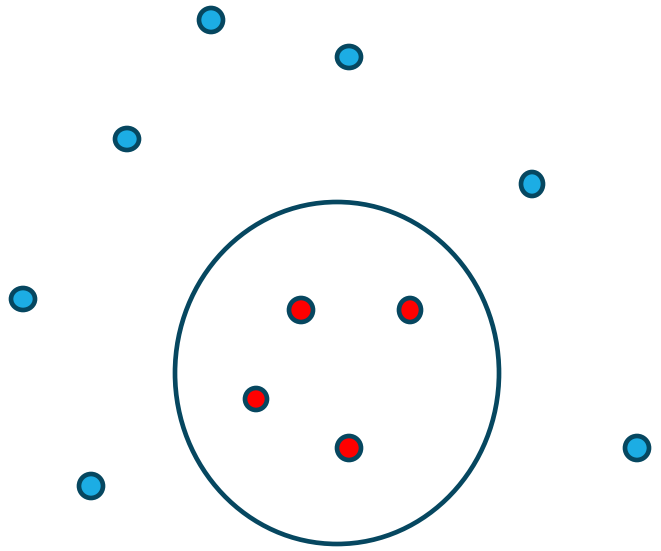




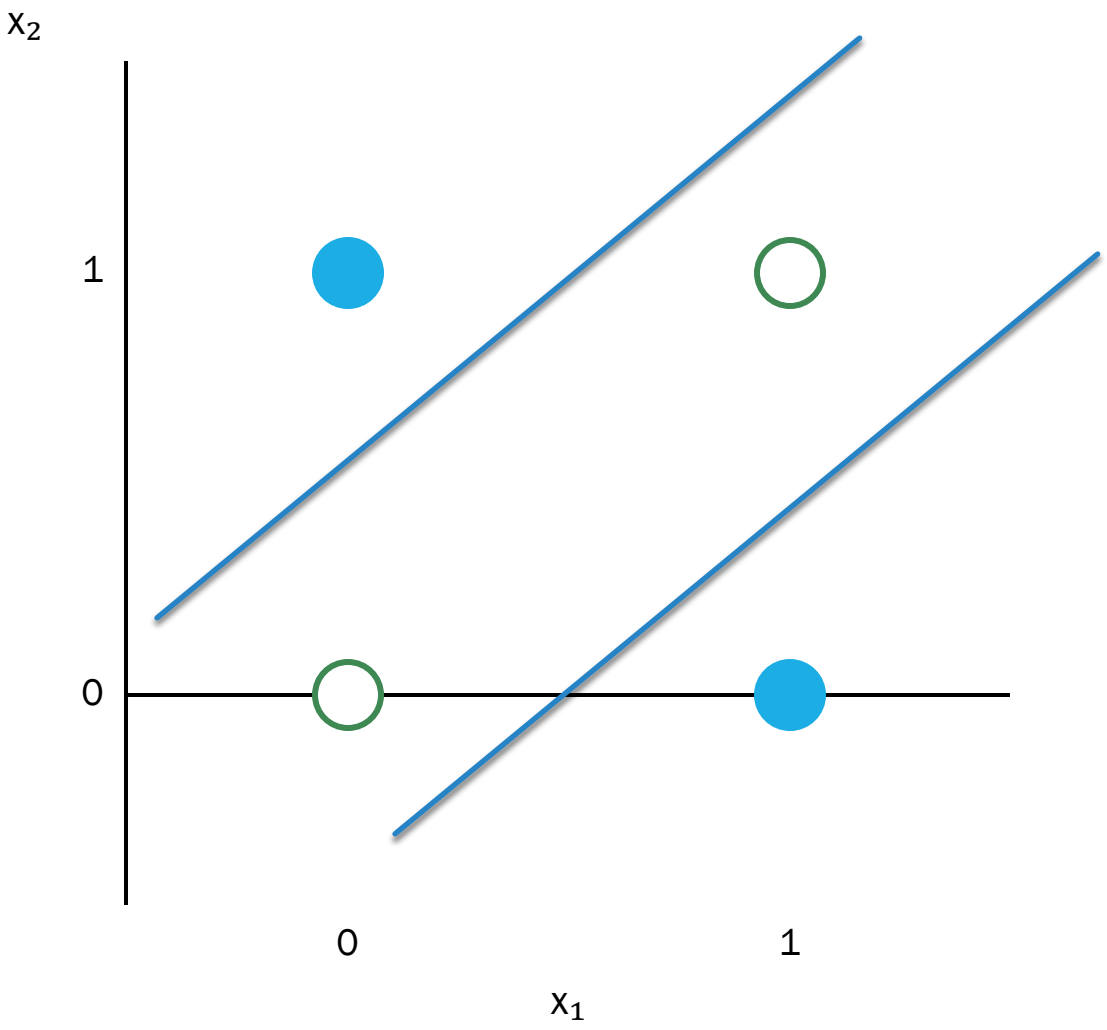
[For reference only] Equation of a plane = $ax + by + cz + d$

Limits of a single-layer perceptron

- A perceptron can solve linearly separable problems
- It can only model linear decision boundaries.
- Can only draw a single line (2 input features), a plane (3 input features), or a hyperplane (3+ input features).
- Fails for:
  - XOR
  - Circular or spiral patterns
  - Any data requiring a curved or disjoint boundary



The XOR challenge



Can one straight line separate filled from open?

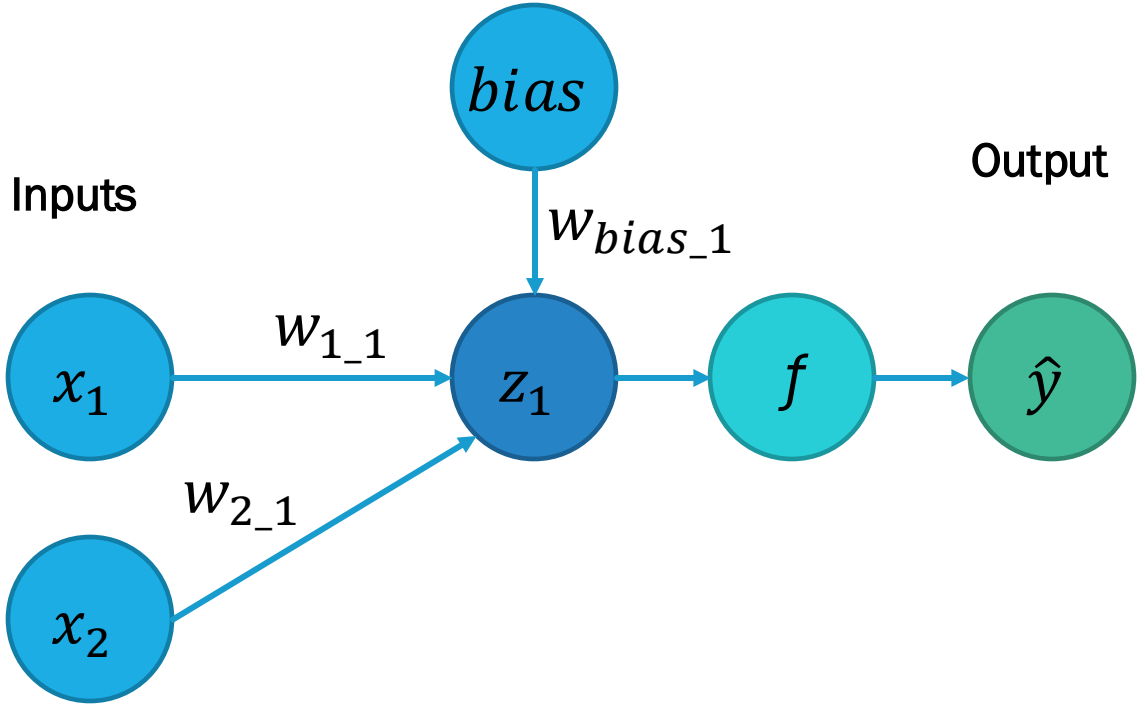
Answer: No, a simple single neuron cannot compute XOR.

Can two straight lines separate them?

○ output 0 ● output 1

x_1, x_2	OR	AND	Output
0, 0	0	0	0
0, 1	1	0	1
1, 0	1	0	1
1, 1	1	1	0

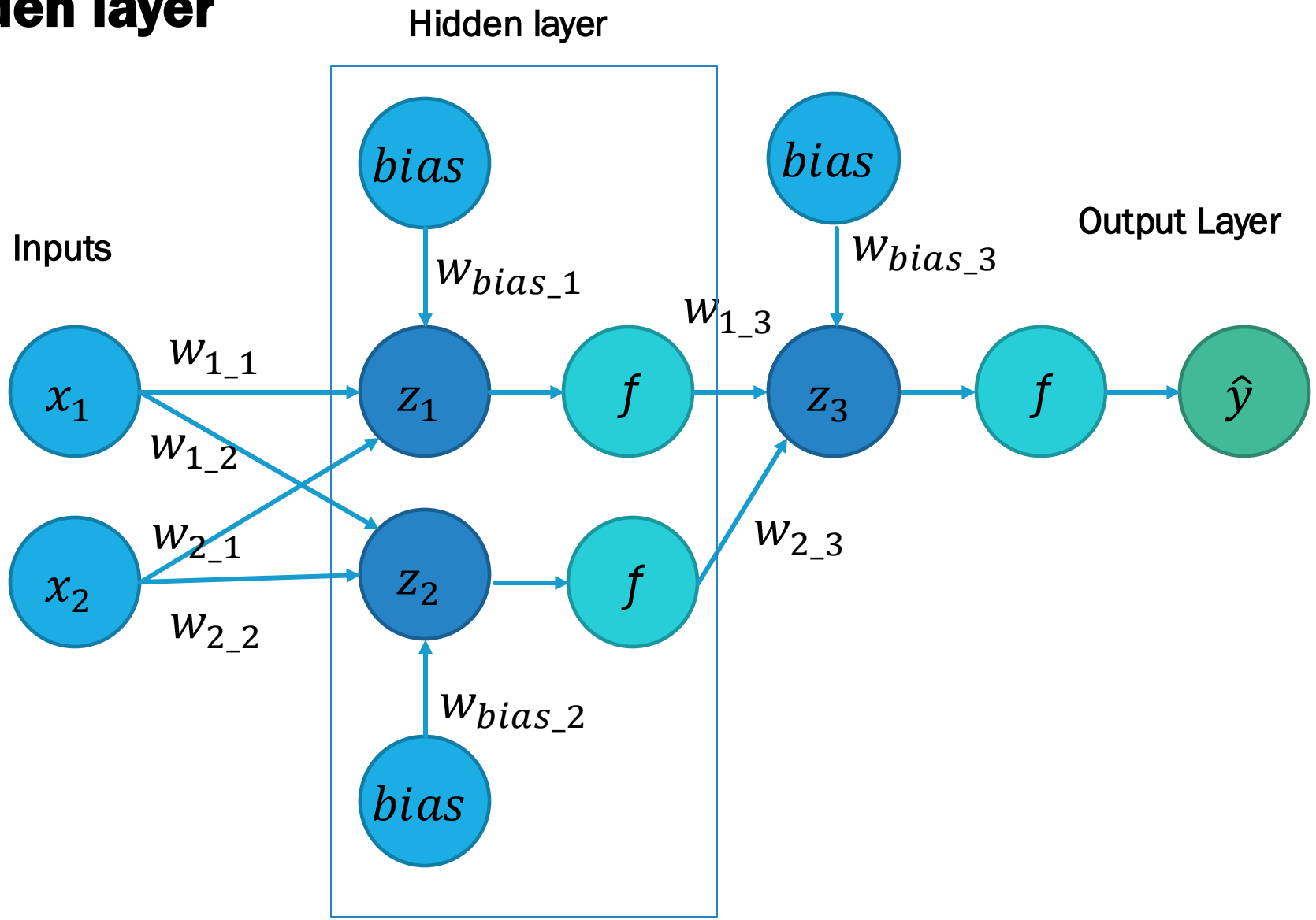
Our perceptron (1 neuron)



One neuron draws one line.

How do we draw two lines?

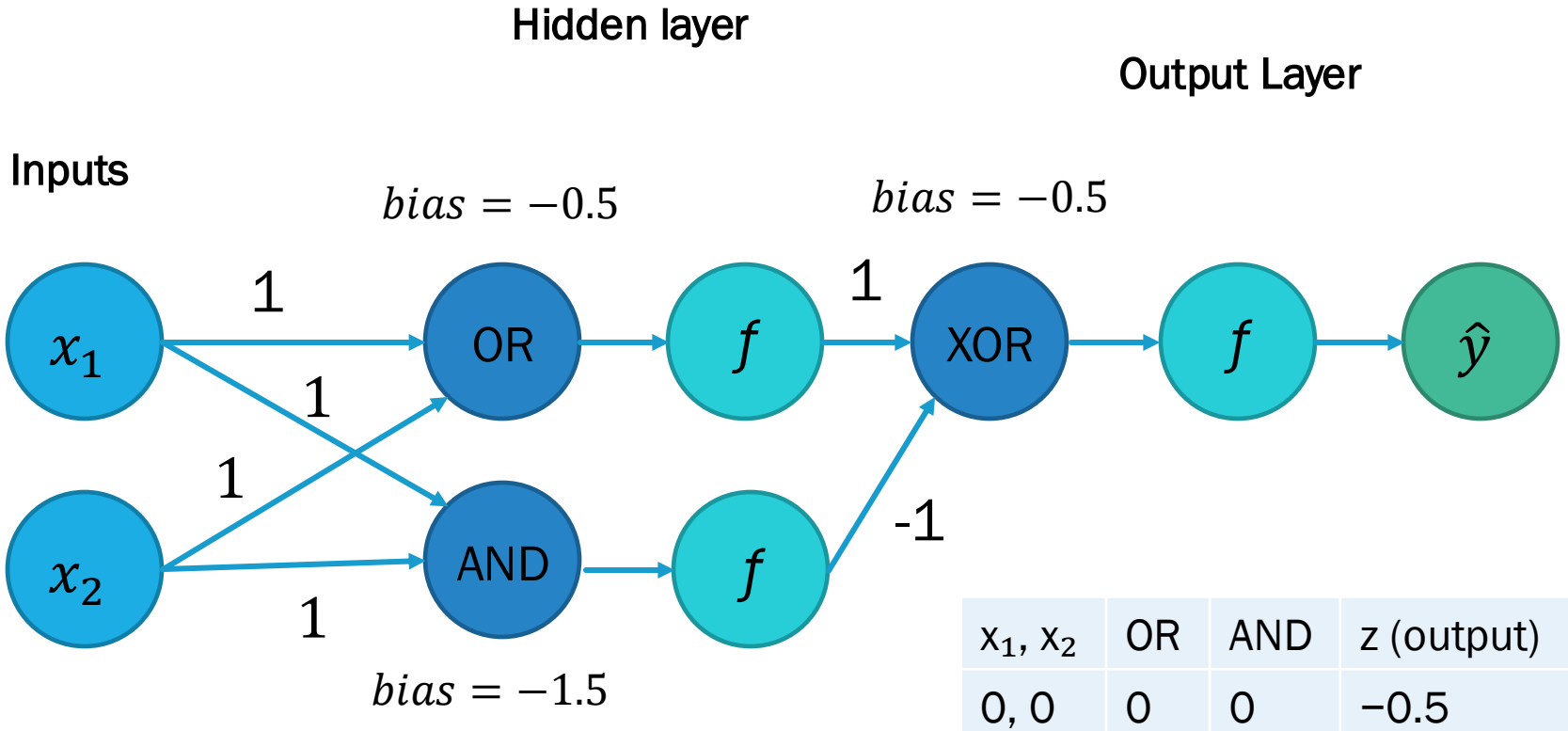
Adding a hidden layer



z = weighted sum
f = activation
$\hat{y} = f(z_3)$

Can we compute XOR?

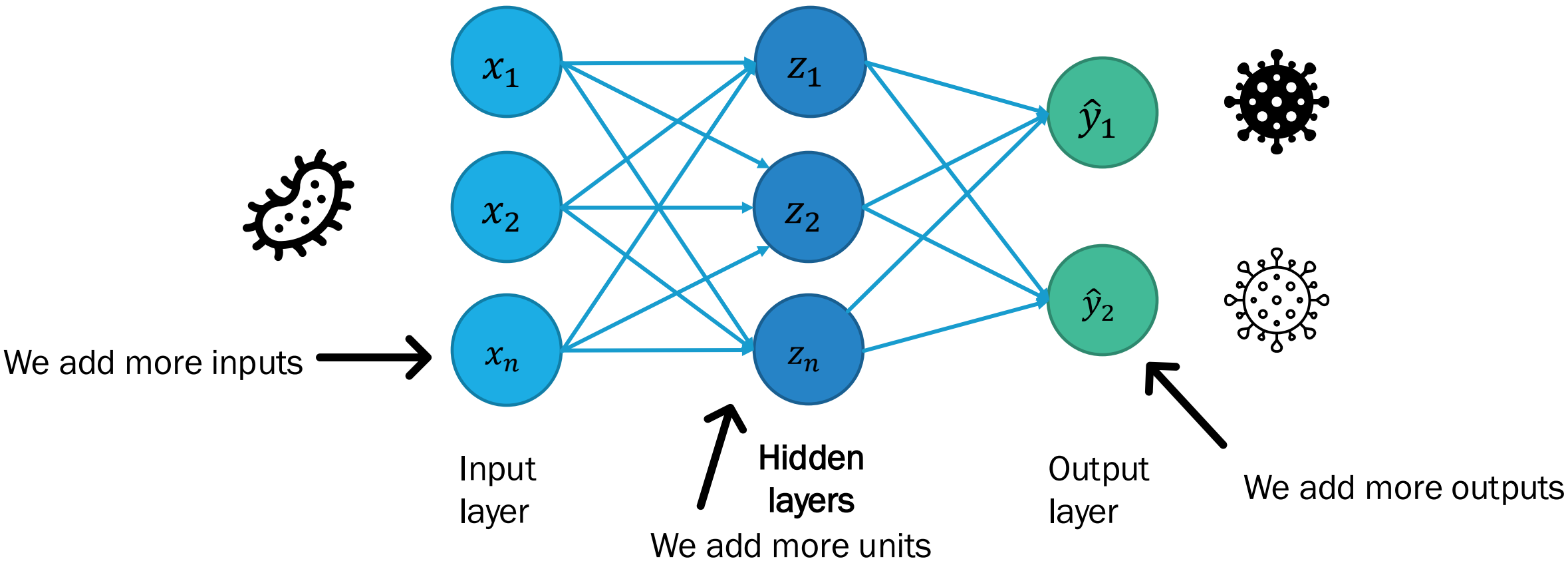
Yes, we can!



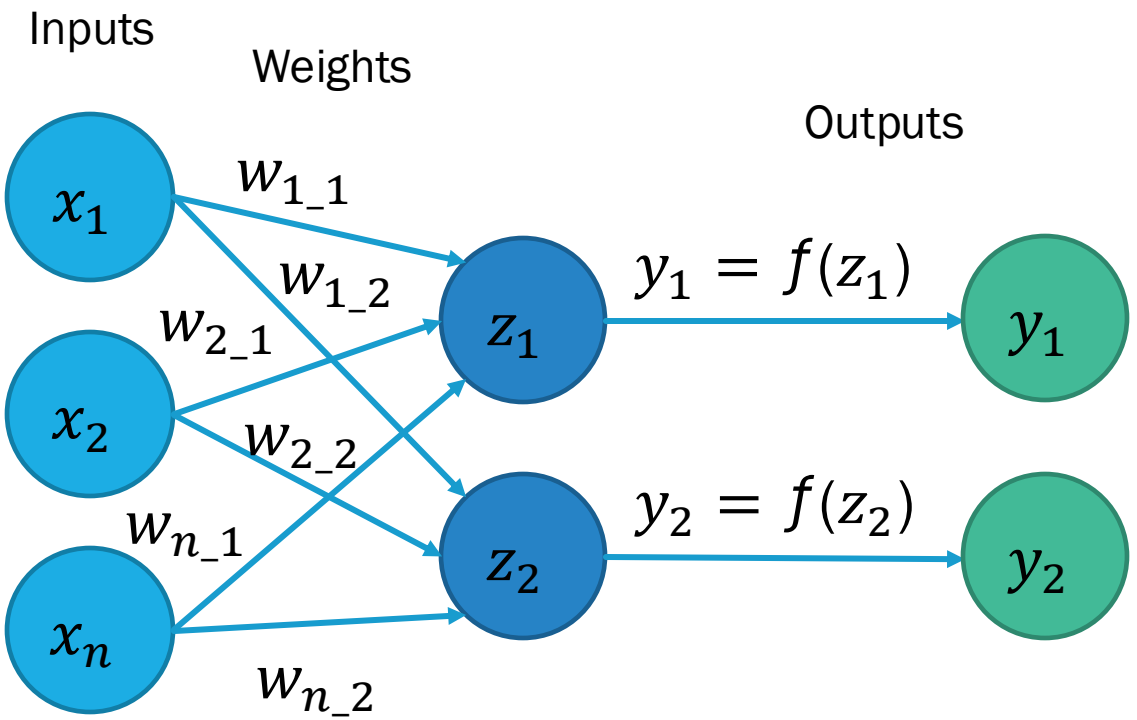
x_1, x_2	OR	AND	z (output)	$\hat{y}$
0, 0	0	0	−0.5	0
0, 1	1	0	+0.5	1
1, 0	1	0	+0.5	1
1, 1	1	1	−0.5	0

Activation function:
f(z) = 1 if z > 0,
otherwise 0

From perceptron to multi-layer neural networks



Artificial Neural networks (ANN)



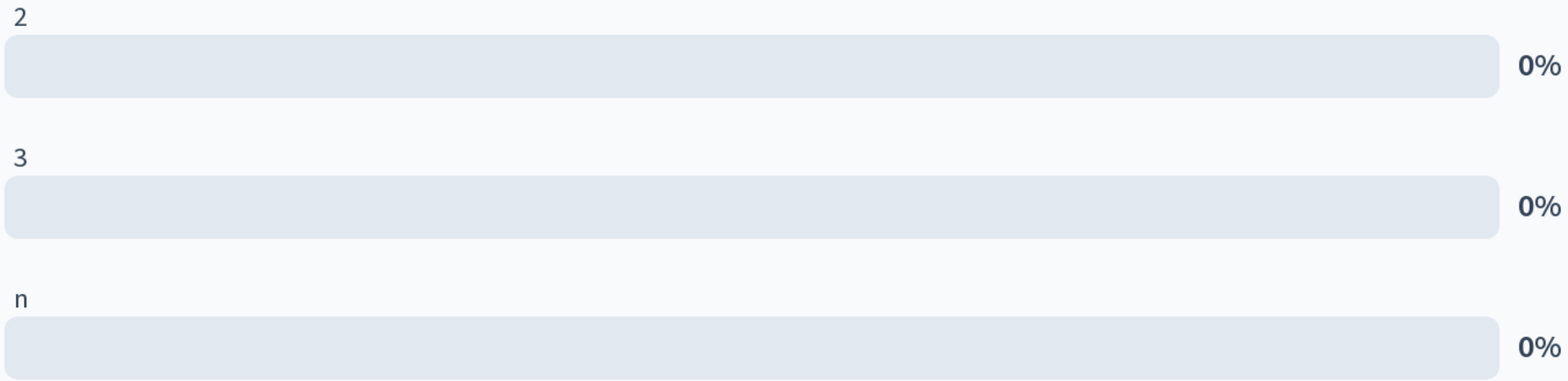
Each output has its own weight for each input

$$z = w_0 + \sum_{i=1}^{n} x_i \, w_i$$

$$\hat{y} = f\left(w_0 + \sum_{i=1}^{n} x_i \, w_i \right)$$

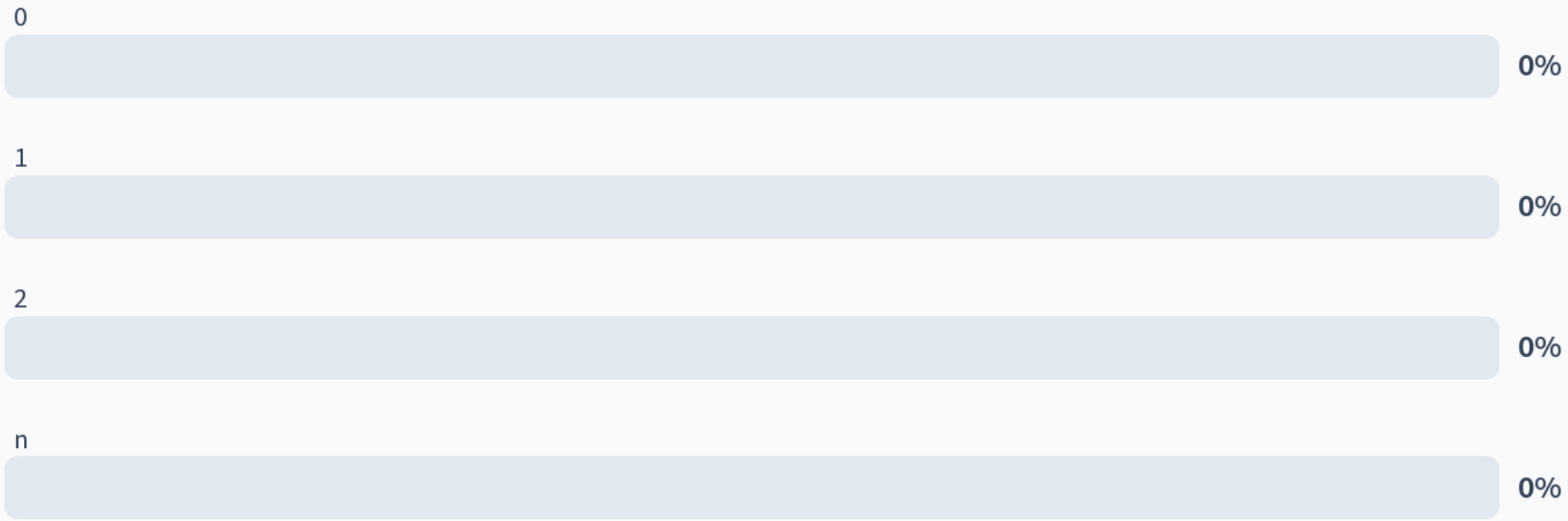


How many inputs can a perceptron have?



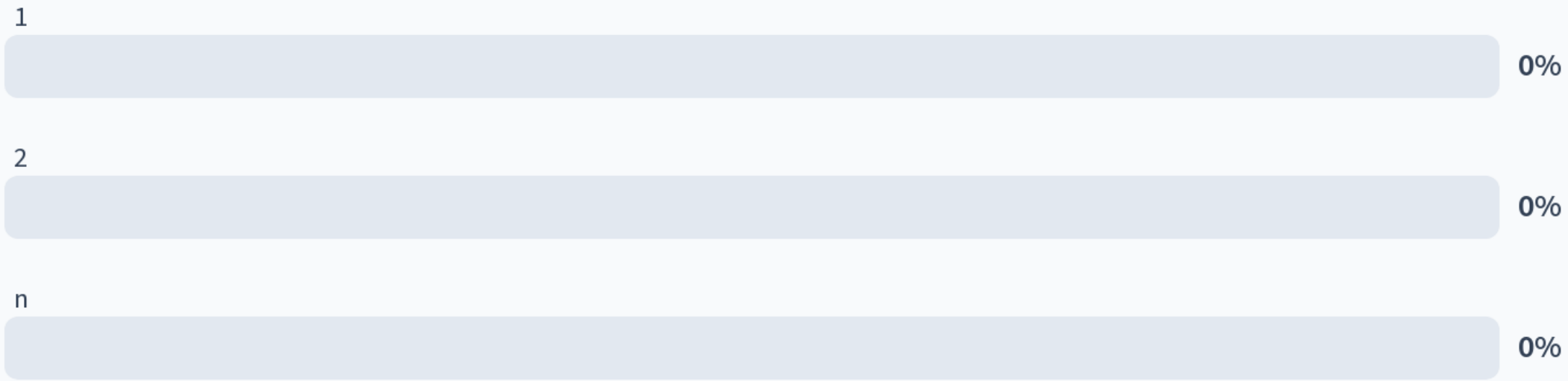
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How many weights has each input in a perceptron?



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How many hidden layers can a network have?



Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Multi-layer neural networks example

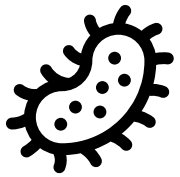
Input

Input layer

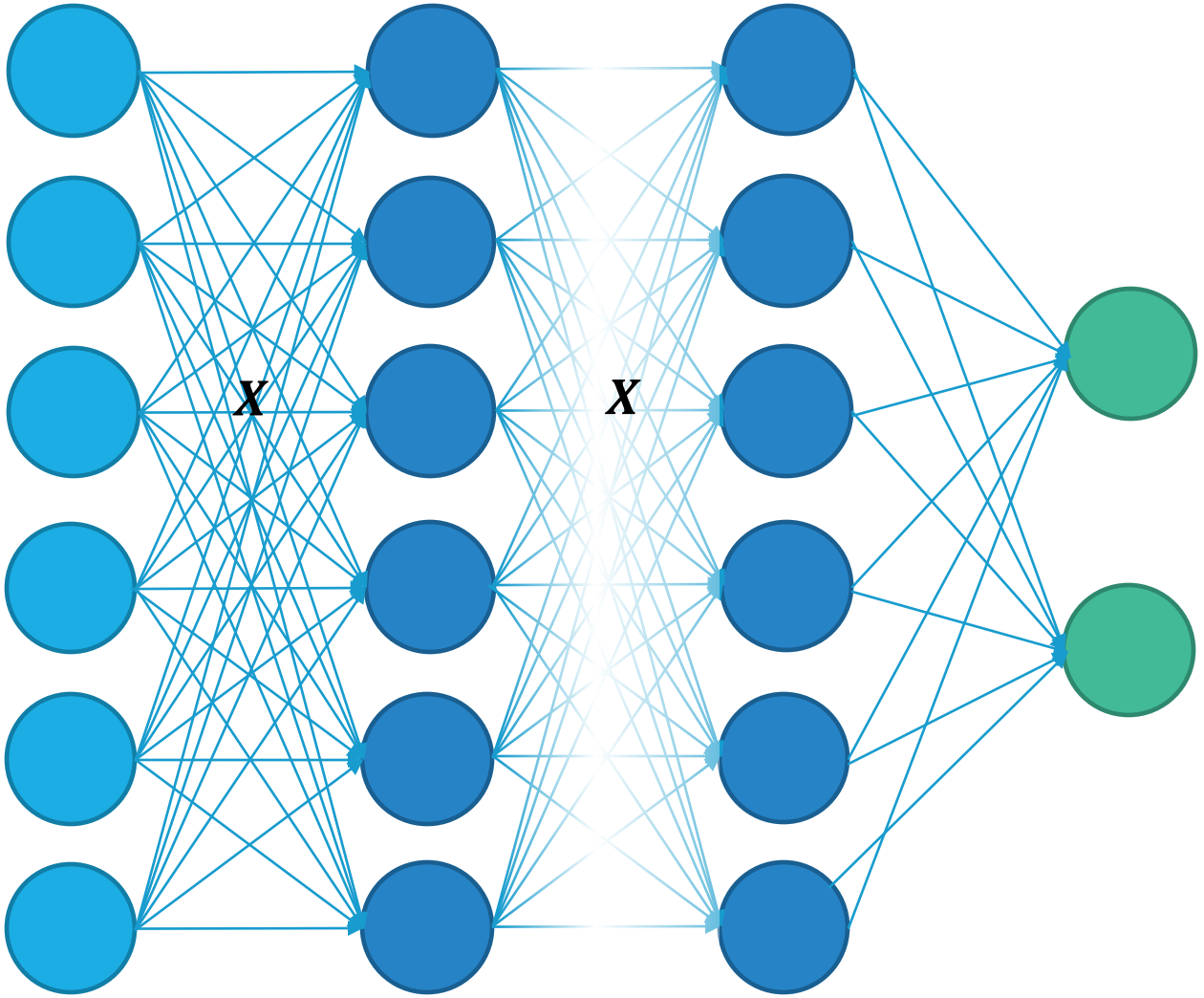
Hidden layers

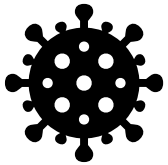
Output layer

Dense layer
==
Fully connected layer



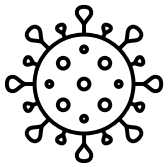
data





Probability of cancer 95%

Probability of no cancer 5%



ANNs: How many inputs and outputs do we need?

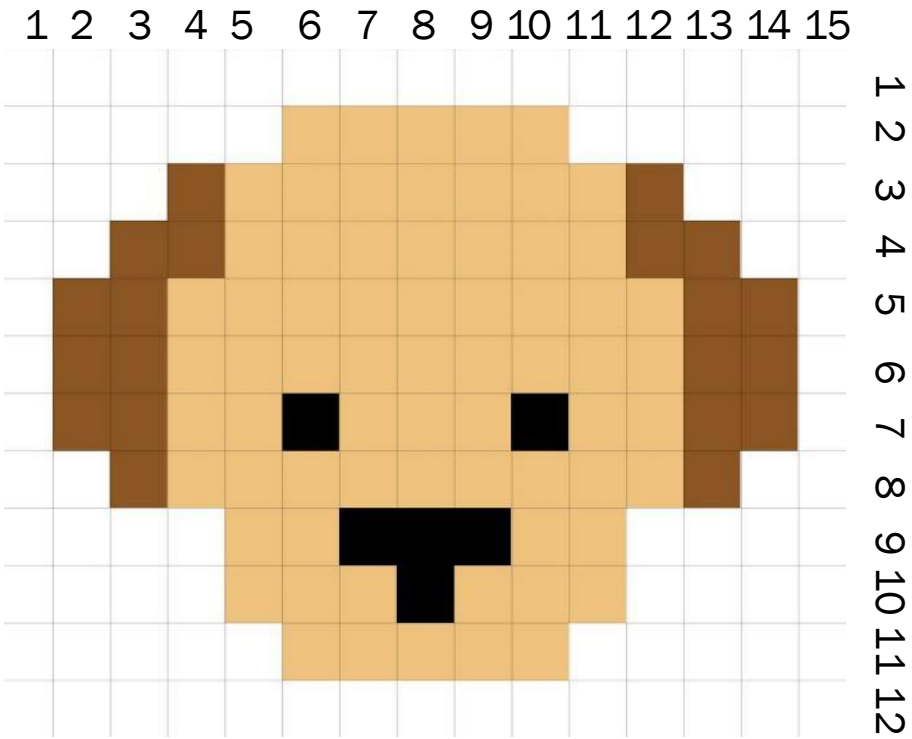
- The number of inputs will depend on our data.
- Example 1: Iris dataset

4 inputs | 1 output

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
50	7.0	3.2	4.7	1.4	versicolor
51	6.4	3.2	4.5	1.5	versicolor
52	6.9	3.1	4.9	1.5	versicolor
53	5.5	2.3	4.0	1.3	versicolor
54	6.5	2.8	4.6	1.5	versicolor
55	5.7	2.8	4.5	1.3	versicolor

ANNs: How many inputs and outputs do we need?

- Example 2: Images



→ 15x12 = 180 pixels → 180 input units

Dog/no dog → 1 label/class to predict → 1 output (0 or 1)

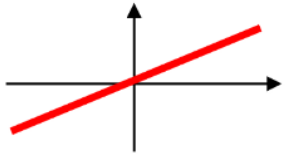
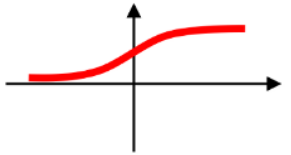
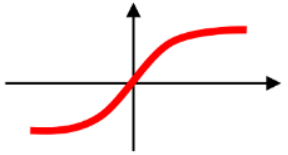
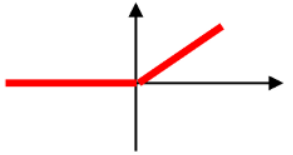
Next week: Convolutional Neural Networks

How many hidden layers and neurons do we need?

- The number of hidden layers, and the number of units depends on the complexity of the problem and there is not strict rule on determining the number of hidden layers.
- The more complex the problem, the more layers and units you will need.
- Easy problem: classify the iris dataset (4 input variables, 1 class as output)
- Complex problem: image recognition (225X255 images, 1000 classes as output)
- For reference:

Model	Layers	Parameters
GPT-3 (OpenAI, 2020 paper)	96 transformer layers	175 billion

For reference only: activation functions

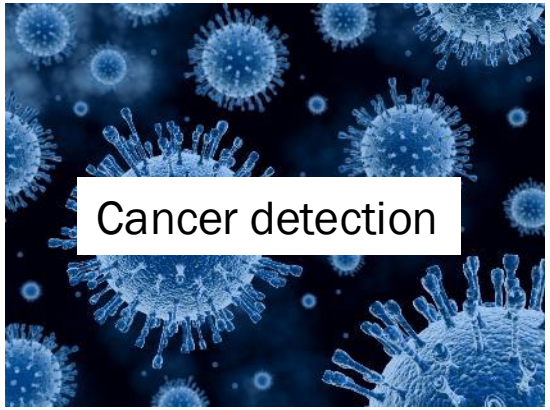
Activation function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent (Tanh)	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	

Copyright © Sebastian Raschka 2016
(http://sebastianraschka.com)

Non-linear

Non-linear activation functions allow to learn more complex mappings

What can we do with Ann?


Cancer detection

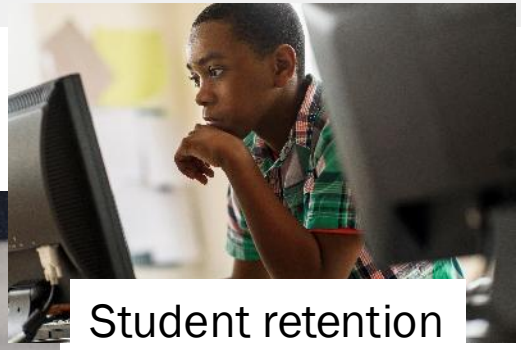
Shopping prediction




Recognition/Classification



Interest prediction


Student retention

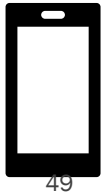

Chatbots


Drug discovery


Weather prediction


Stock market prediction

Do you know more examples?



Research

Forecasting Climatic Trends Using Neural Networks: An Experimental Study Using Global Historical Data

Takeshi Ise [1,2] and Yurika Oba [1]*

[1] Field Science Education and Research Center (FSERC), Kyoto University, Kyoto, Japan, [2] Japan Science and Technology Agency (JST), Kawaguchi, Japan

Review Article

Artificial neural networks in the cancer genomics frontier

Andrew Oustimov[1], Vincent Vu[2]

[1]Department of Epidemiology & Biostatistics, College of Public Health, University of South Florida, Tampa, FL 33620, USA; [2]Department of Mathematics and Statistics, University of California, Los Angeles, CA, USA

Correspondence to: Andrew Oustimov, MPH. Department of Epidemiology & Biostatistics, College of Public Health, University of South Florida, 13201 Bruce B Downs Blvd, Tampa, FL 33620, USA. Email: aoustimo@mail.usf.edu.

IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, VOL. 32, NO. 10, OCTOBER 2021 4291

Attention in Natural Language Processing

Andrea Galassi, Marco Lippi, and Paolo Torroni

AUTOMATIC LANGUAGE IDENTIFICATION USING DEEP NEURAL NETWORKS

Ignacio Lopez-Moreno[1], Javier Gonzalez-Dominguez[1,2], Oldrich Plchot[3], David Martinez[4], Joaquin Gonzalez-Rodriguez[2], Pedro Moreno[1]

[1]Google Inc., New York, USA
[2]ATVS-Biometric Recognition Group, Universidad Autonoma de Madrid, Spain
[3]Brno University of Technology, Czech Republic
[4]Aragon Institute for Engineering Research (I3A), University of Zaragoza, Spain
{elnota,jgd}@google.com

Tran *et al. Genome Medicine* (2021) 13:152
https://doi.org/10.1186/s13073-021-00968-x

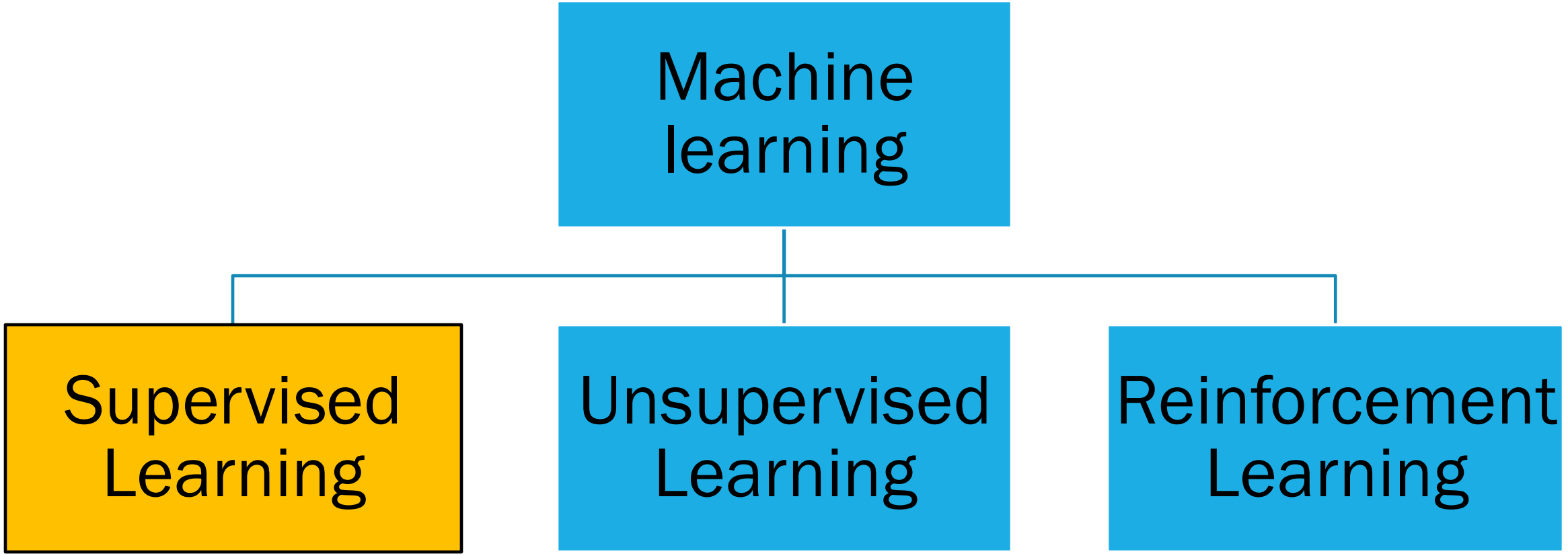
Genome Medicine

REVIEW Open Access

Deep learning in cancer diagnosis, prognosis and treatment selection

Khoa A. Tran[1,2], Olga Kondrashova[1], Andrew Bradley[4], Elizabeth D. Williams[2,3], John V. Pearson[1] and Nicola Waddell[1*]

RECAP: Types of machine Learning

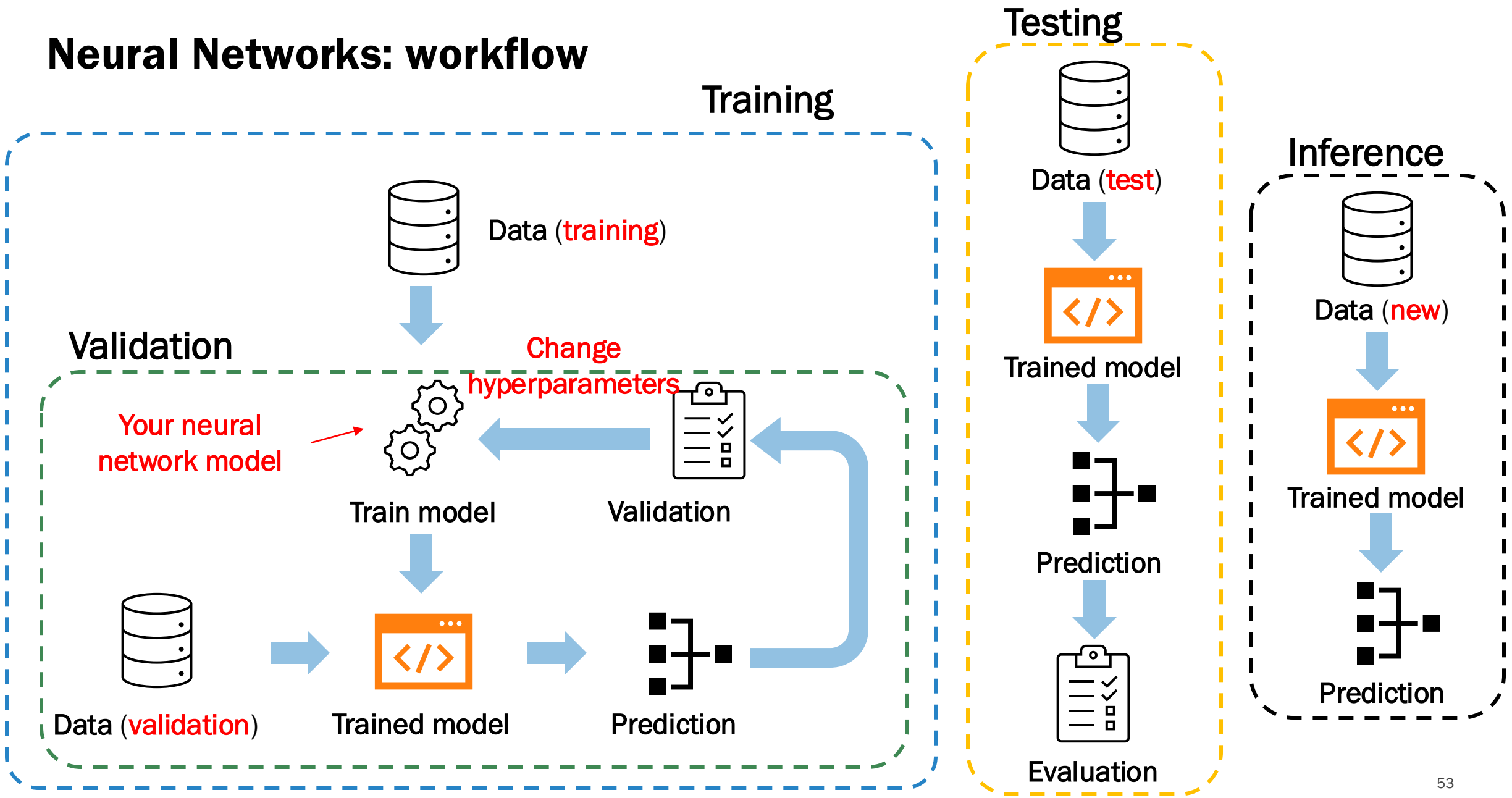


We have both **supervised** and **unsupervised** neural networks.
We will focus the rest of the lecture on **supervised** learning.

Next…

- How to train neural networks
- Main issues during training and how to solve them.
- Data: data split, data bias, …
- Evaluation

Neural Networks: workflow



Data SPLIT

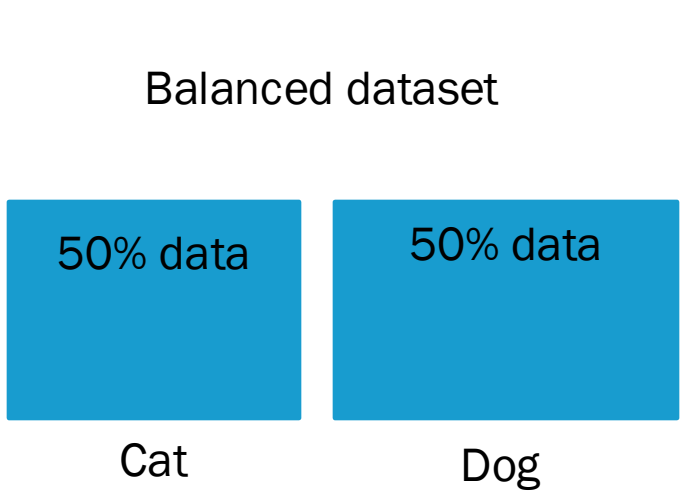
- Dataset:

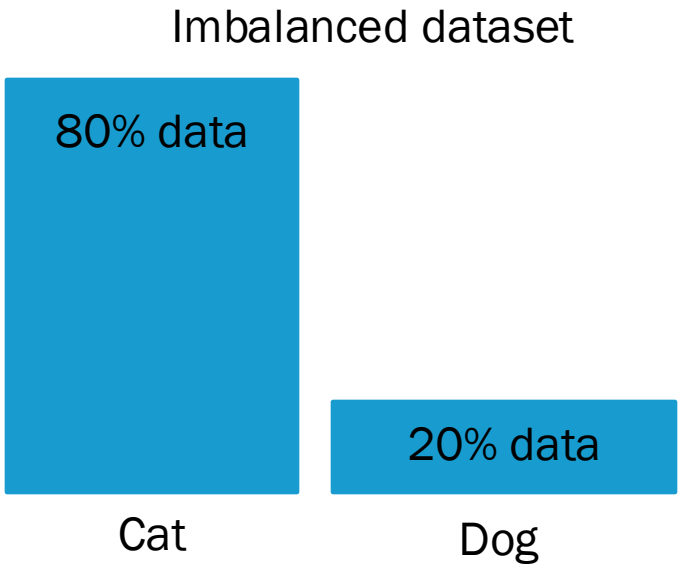


80% 10% 10% (larger datasets)

60% 20% 20% (small datasets)

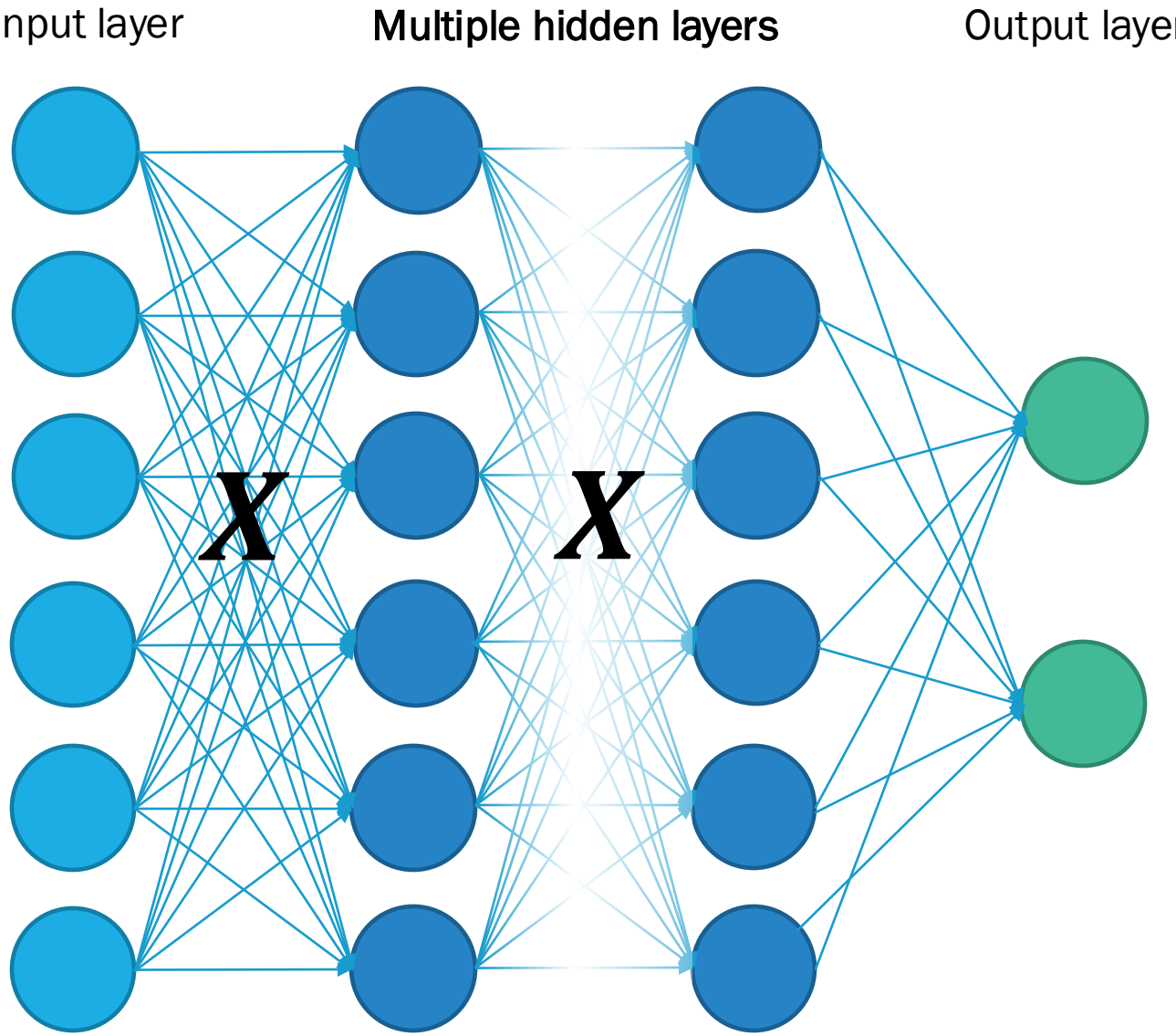
- Know your data:





Neural network example

Input



How do we initialise weights?

How can we find the optimum weights so that the error is minimal?

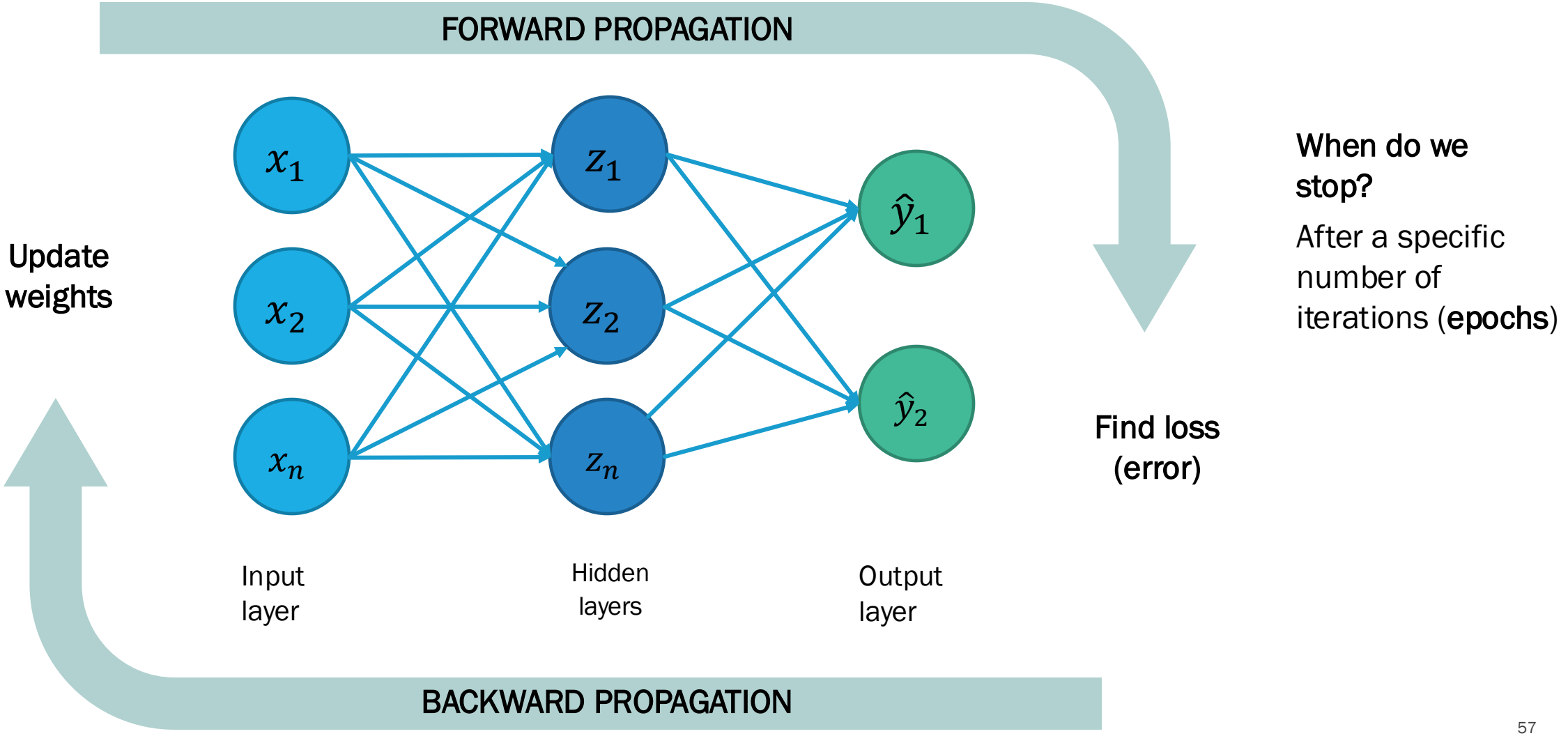
Training a neural network

Train model

- Objective: find the optimum weights that will give the lowest **error** [**loss**]
- Optimisation (Gradient Descent)
  1. Randomly initialise weights
  2. Find how much the weights need to change (by computing the derivatives)
  3. Update weights by taking a small step [**learning rate**]
  4. Repeat until convergence (until there is no improvement) [**number of epochs**]

Training a neural network: loop

Train model



When do we stop?

After a specific number of iterations (**epochs**)

For REFERENCE only: Loss optimisation

Train model

- **Identify (train) a set of weights that will give us the minimum error**
- Loss functions: Cross entropy, Mean Squared Error, Mean Absolute Error.
- Needs to be differentiable (the derivative of the function exists for all points).

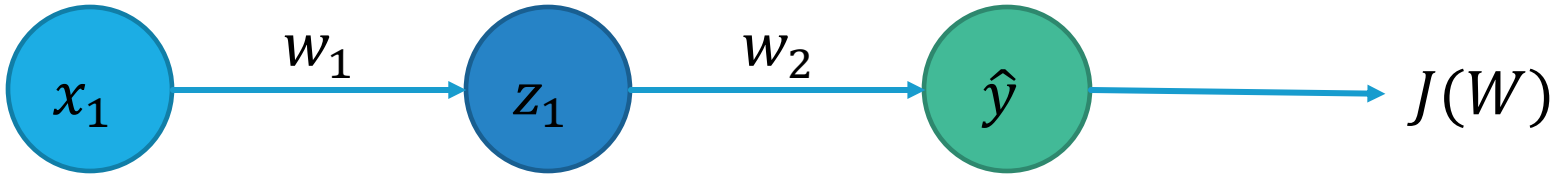
Binary classification	Multiclass classification	Regression
Loss: Cross Entropy	Loss: Cross Entropy	Loss: Mean Squared Error
Activation: Sigmoid	Activation: Softmax	

Backpropagation: Backward propagation of errors

Train model

- Compute the gradients (computationally expensive!)

How does a small change on the weights affect the final loss $J(W)$?



Watch: https://www.youtube.com/watch?v=IN2XmBhILt4&list=PLblh5JKOoLUIxGDQs4LFFD--41Vzf-ME1&index=4

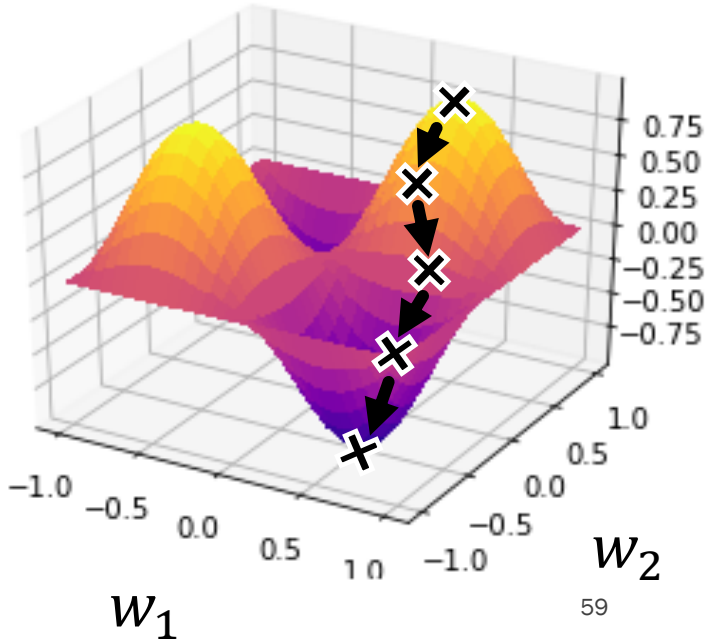
How much a change in w_2 will affect the total error (loss) $J(W)$?

The partial derivative of the loss with respect to w_2

$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

How much a change in w_1 will affect the total error (loss) $J(W)$?

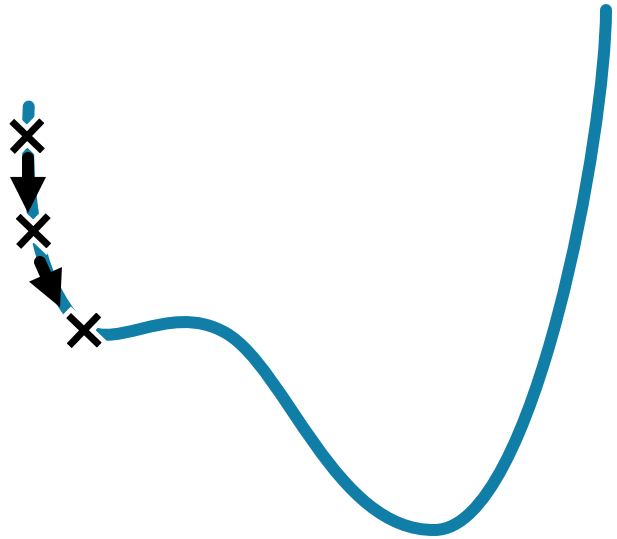
$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$



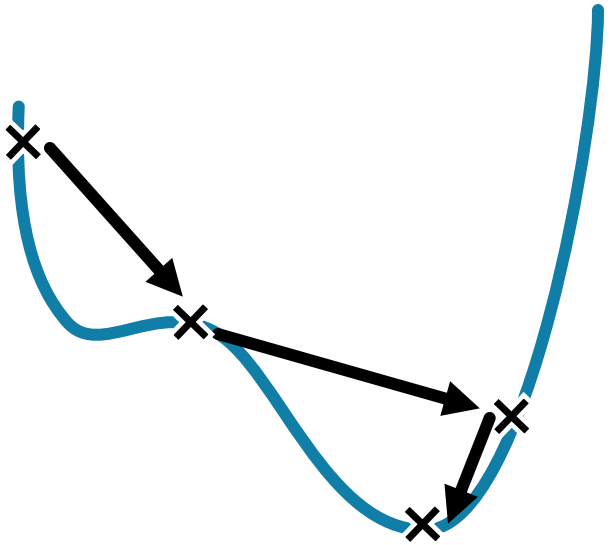
Learning Rate

Train model

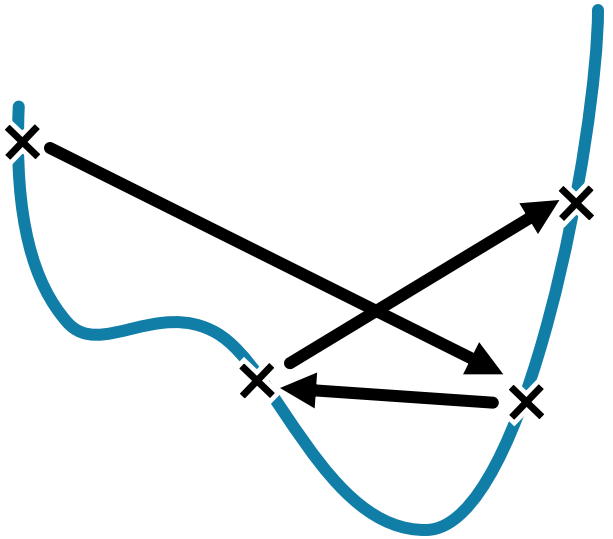
- How much step we take in the gradient?



Small learning rate



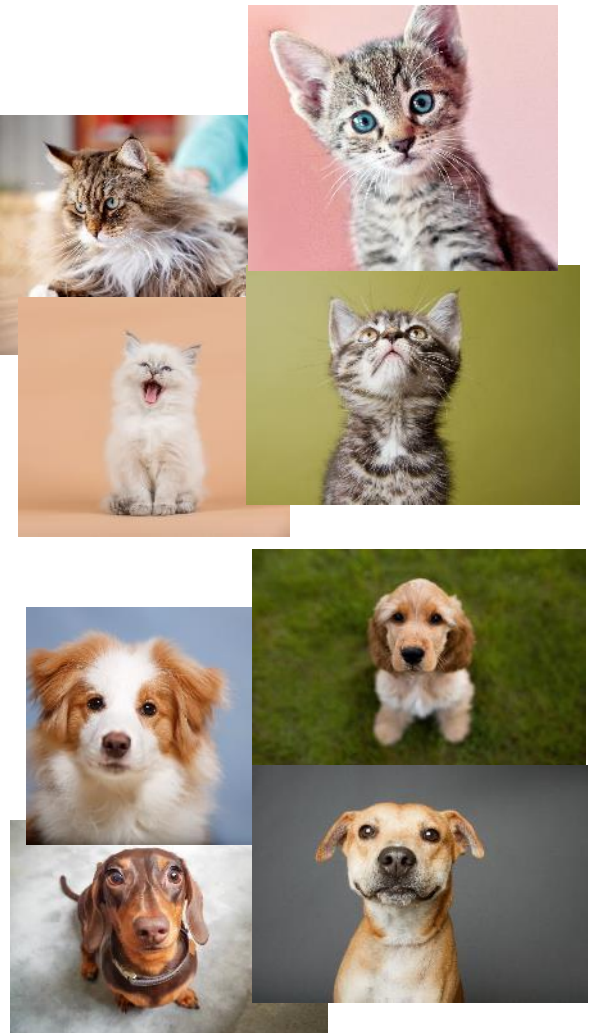
Good learning rate



Large learning rate

Forward propagation and backpropagation

Train model



Input (flatten)

Input layer

Multiple hidden layers

Output layer

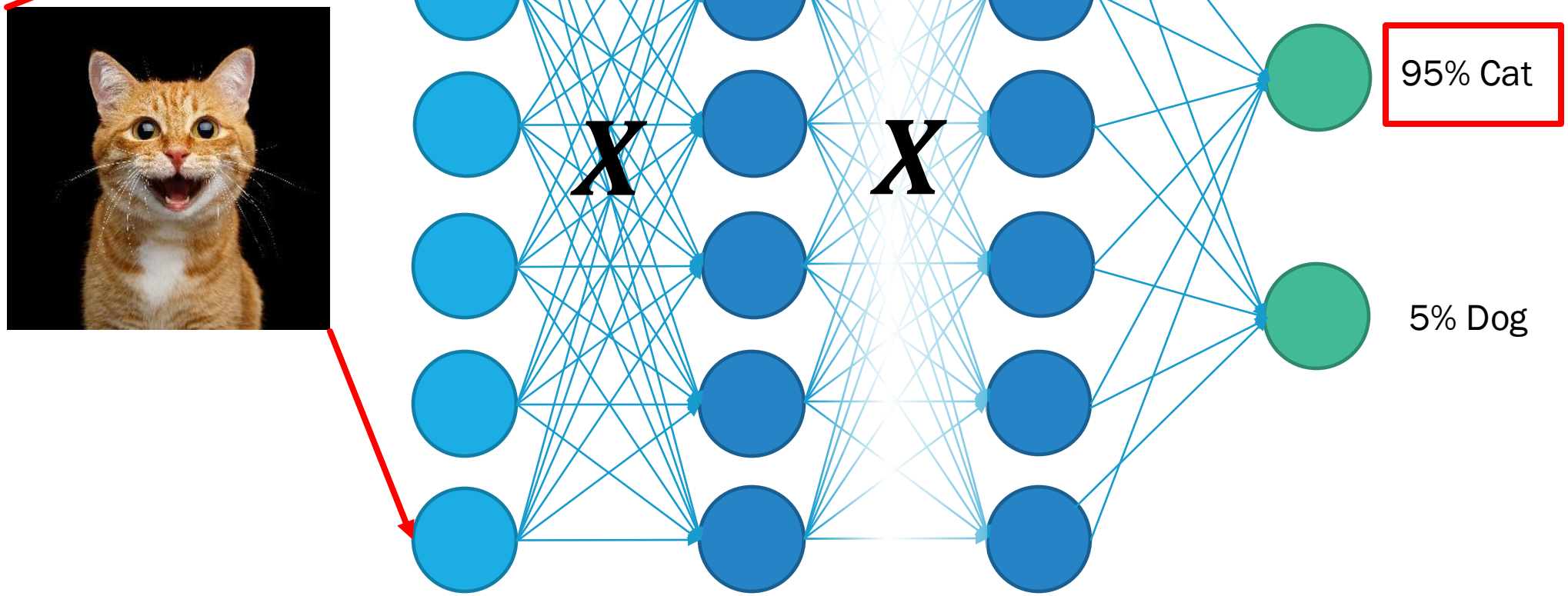
X

X

% Cat

% Dog

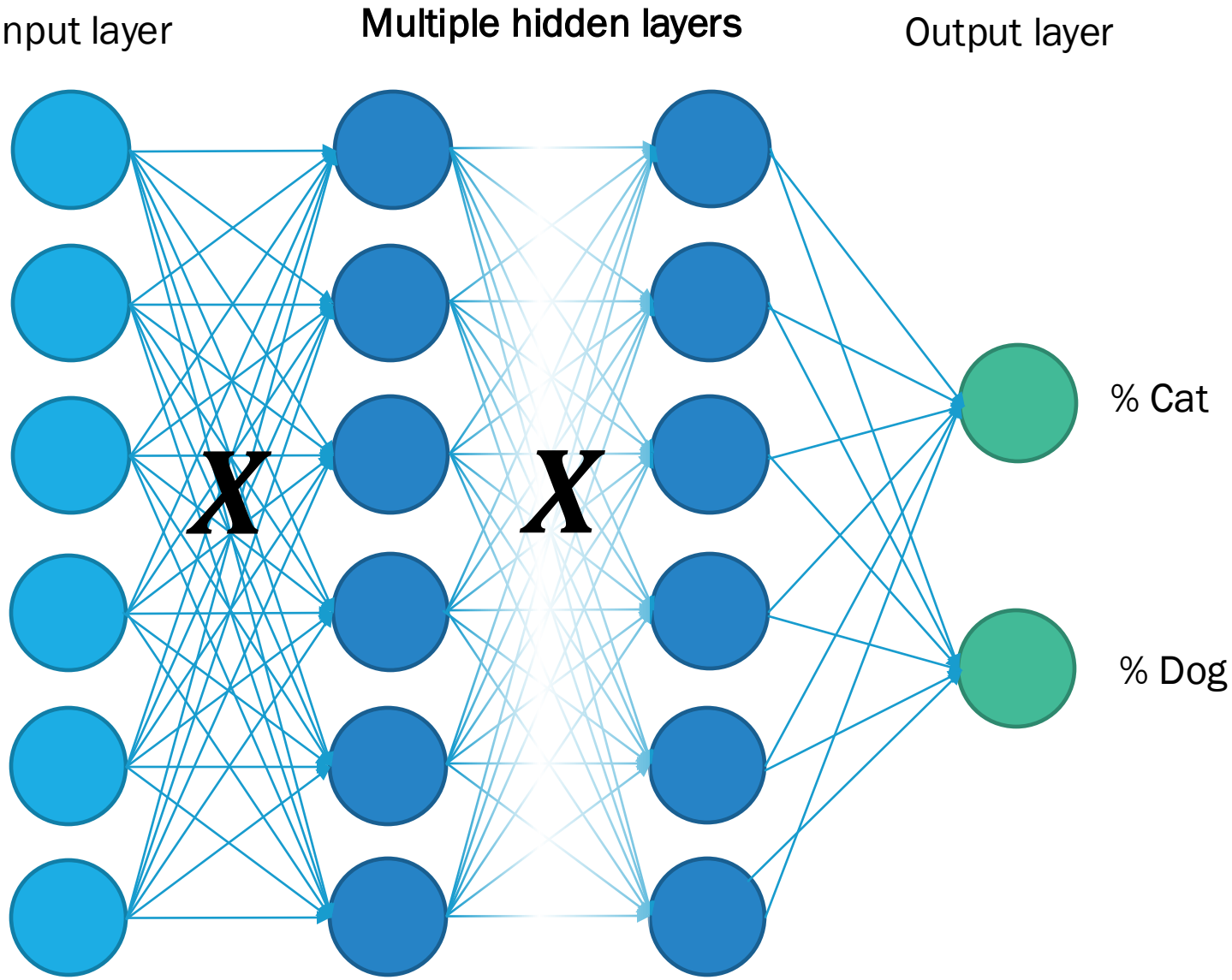
Inference


Input
(flatten)
Input layer
Multiple hidden layers
Output layer
X
X
95% Cat
5% Dog

Inference

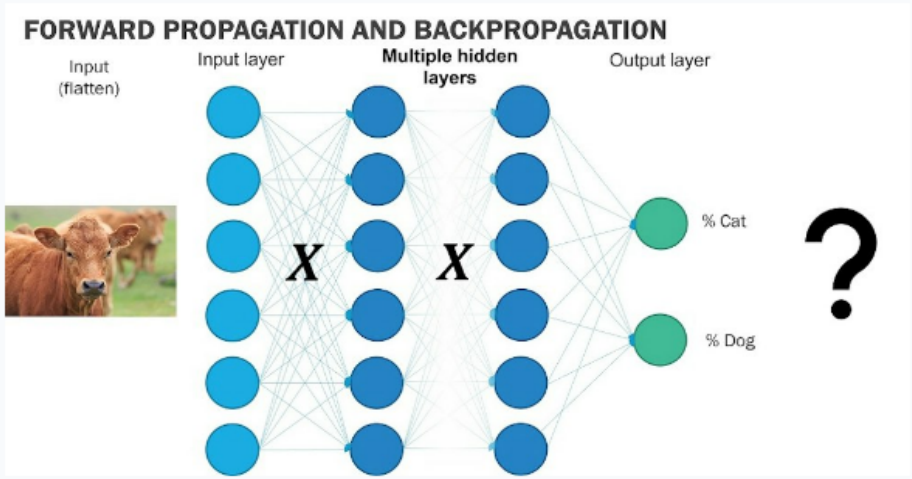
Input
(flatten)





?

What will the NN predict?



Will predict cow

0%

Will predict cat or dog

100%

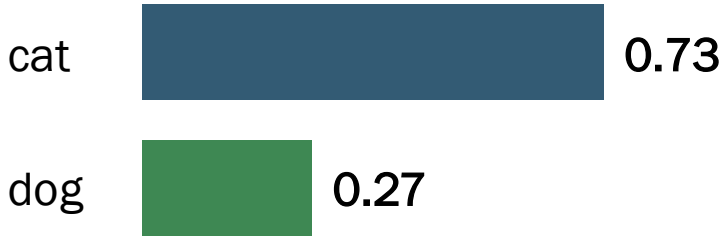
Will not make a prediction

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Softmax and the cow





What should this network answer for a cow?



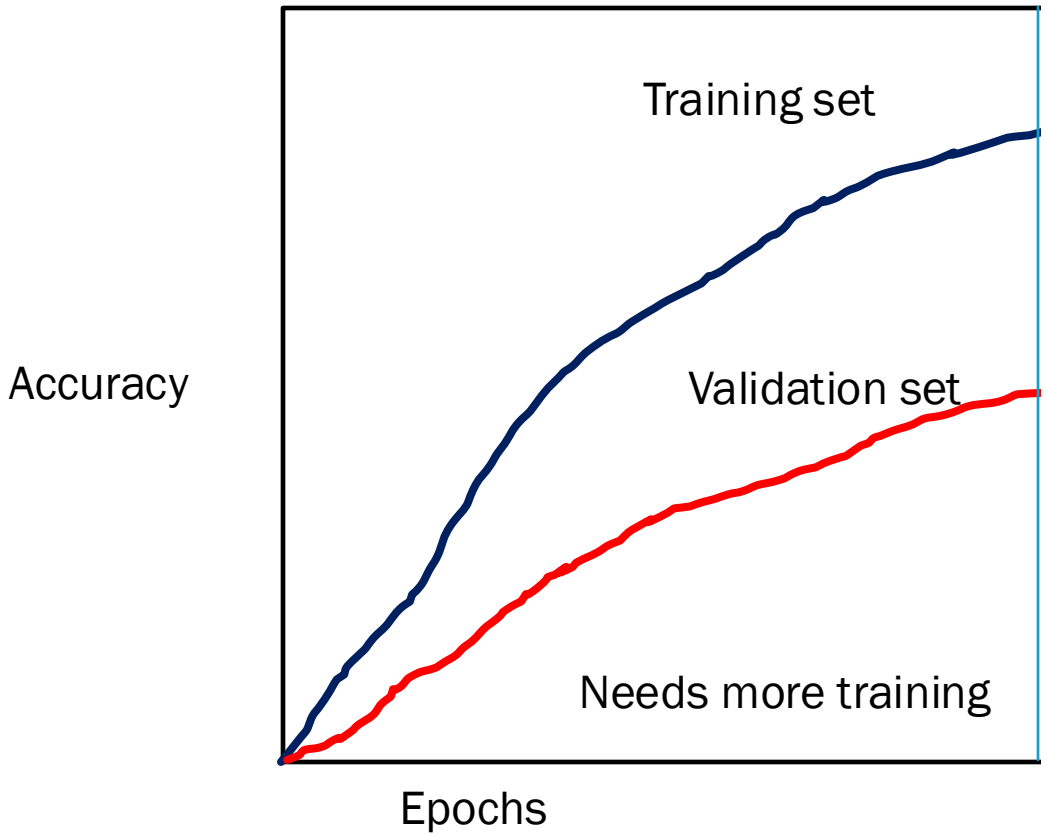


$\text{softmax}(z)_i = e^{z_i} / \Sigma_j e^{z_j}$ the outputs always sum to 1

Training (how long?)

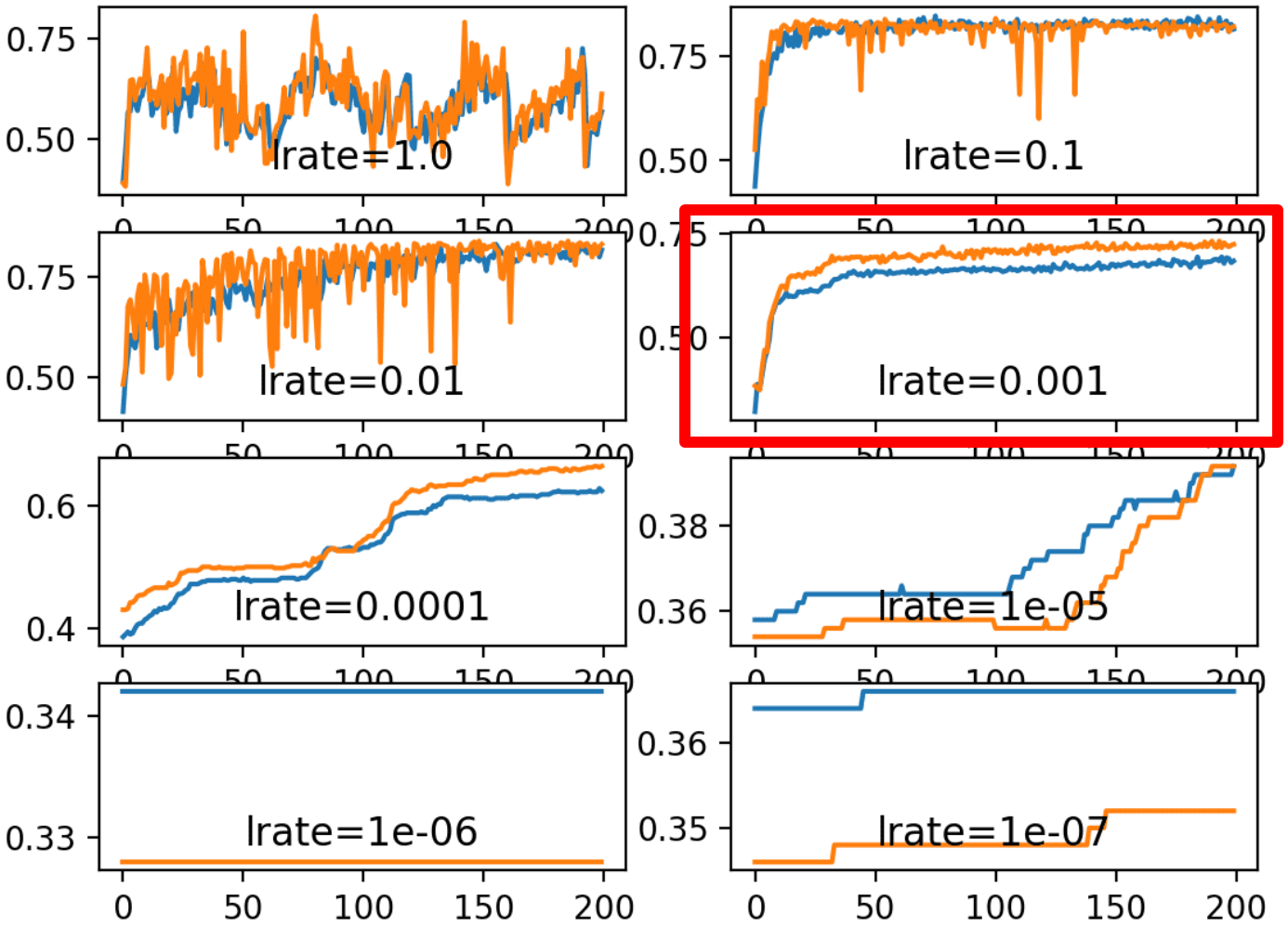
Train model

- Some (most) neural networks will not reach a 100% accuracy. How long do we have to train them?



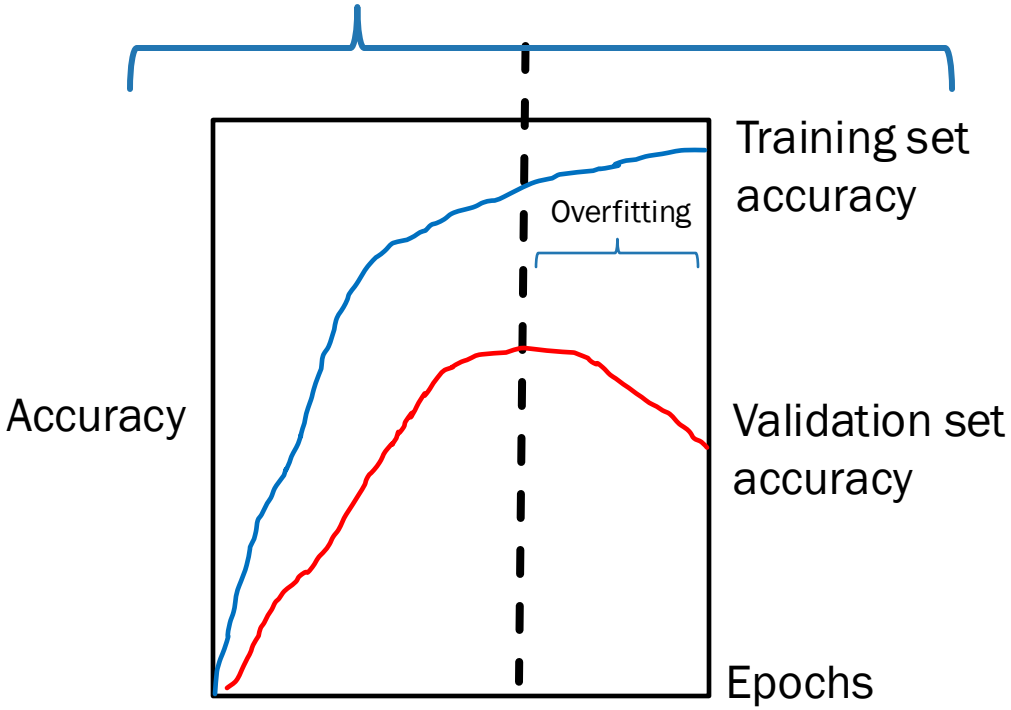
Problems: Learning rate

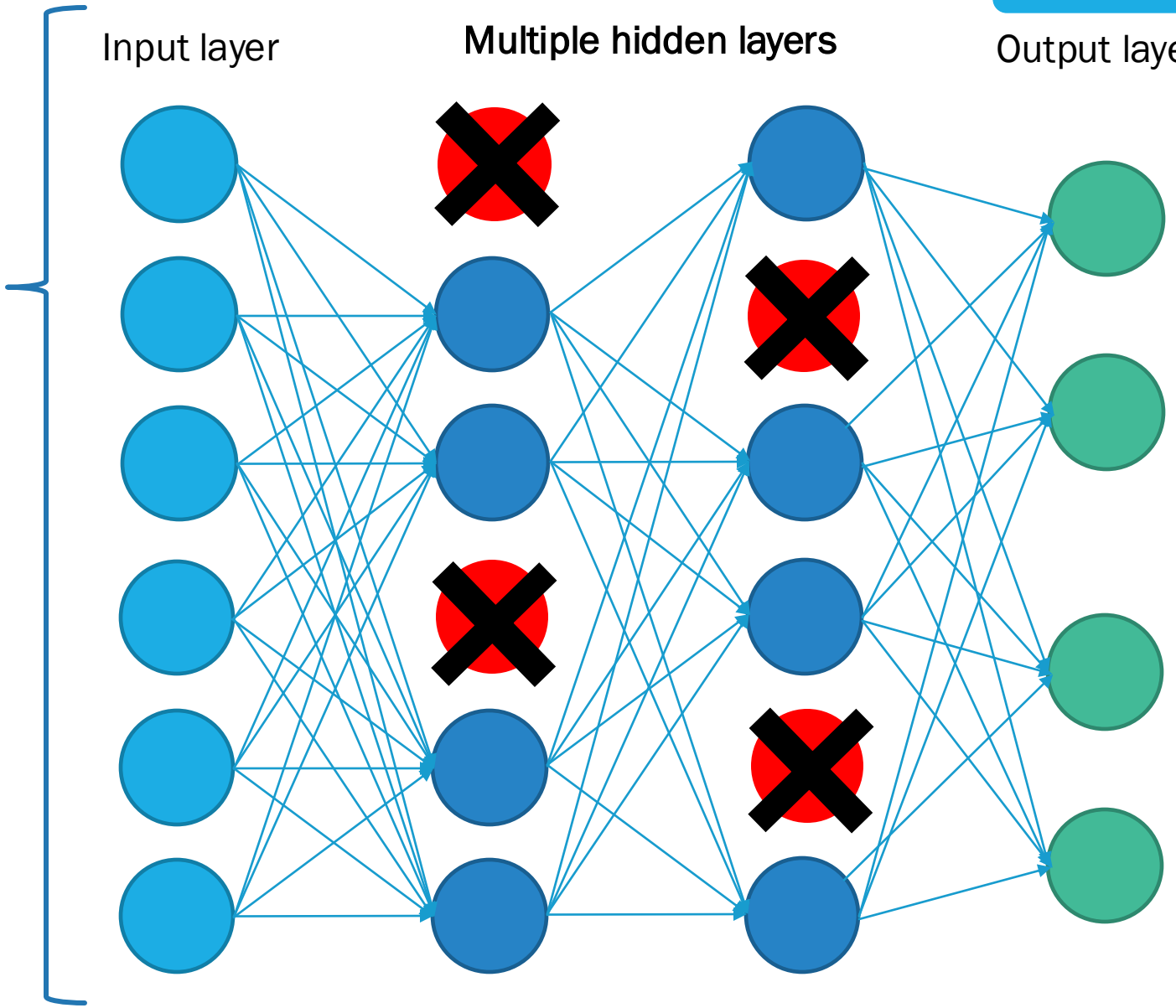
- If the learning rate is too high, the loss can oscillate or diverge
- If the learning rate is too small, the training can get stuck



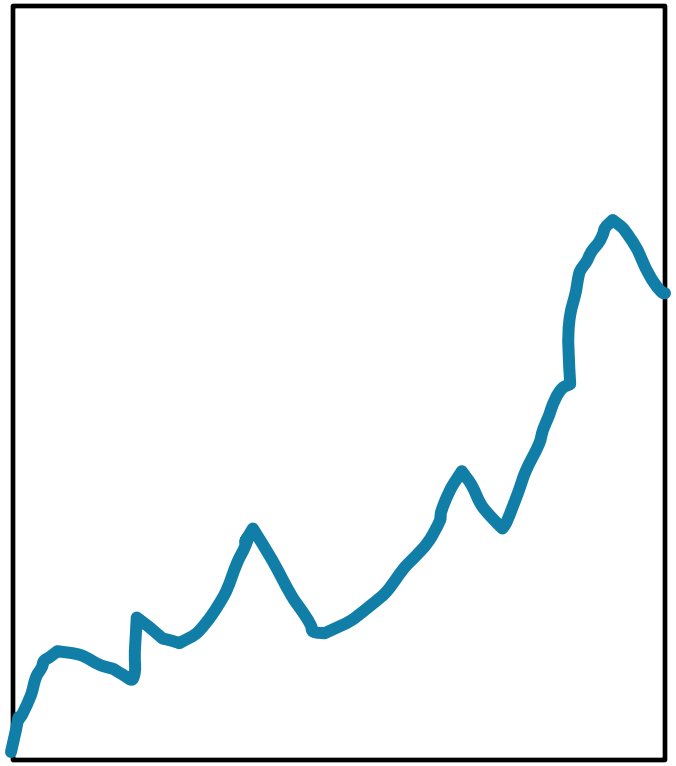
Problems: overfitting

- When the network has 'memorised' the training data. Regularisation techniques:
  - **Dropout:** We set some activation to 0 (usually 0.5) during training
  - **Early stopping**: stop training before it overfits the data





Training





Log in to Poll Everywhere

To present live activities, please log in to your Poll Everywhere account in a separate window.

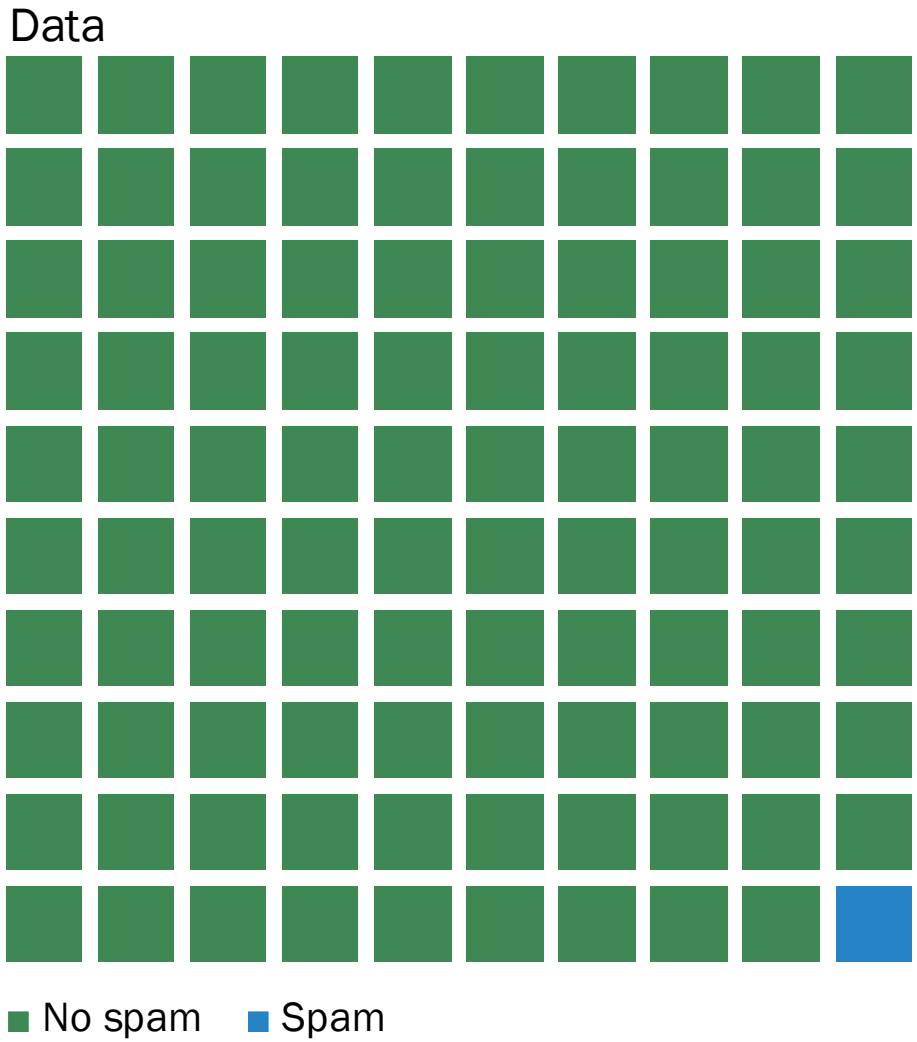
Launch log-in window

How do we know it works?

Our spam detector is tested on 100 emails it has never seen. The accuracy is 99%.

Can we trust it?

Accuracy: how good is the model?



A model says “No spam" every time.
The model never says “spam”.

What is its accuracy?

$$accuracy = \frac{number\ correct\ predictions}{total\ number\ of\ emails}$$

$$accuracy = \frac{99}{100} = \mathbf{0.99\ (99\%!!)}$$

Can we trust it?

Count the mistakes: confusion matrix

	Flagged spam	Passed as good
Really spam (1)	0 caught spam	1 missed spam
Really not spam (99)	0 good email flagged	99 good email passed

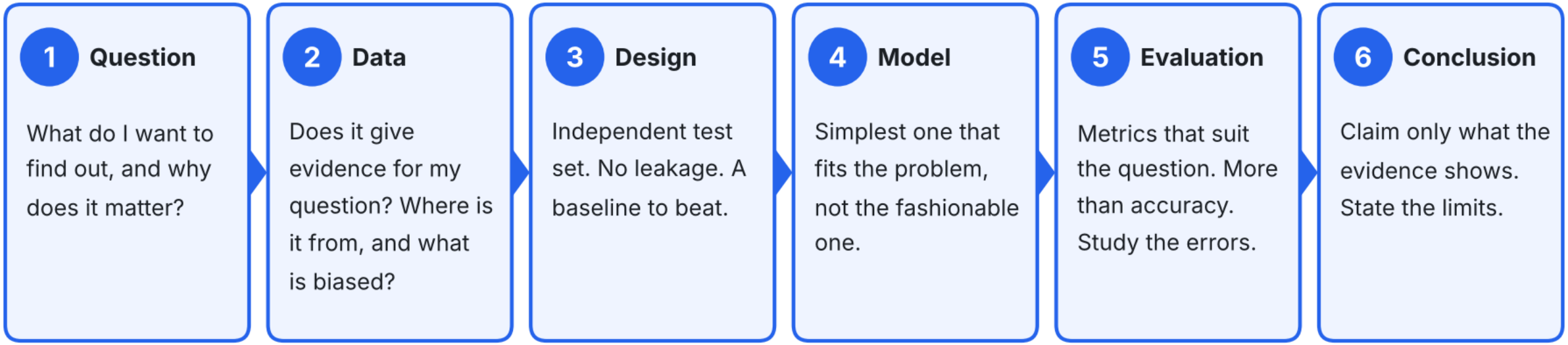
Count the mistakes: confusion matrix

	Flagged spam	Passed as good
Really spam (10)	6 caught spam	4 missed spam
Really not spam (90)	3 good email flagged	87 good email passed

$$accuracy = \frac{(6 + 87)}{100} = \mathbf{0.93}\ (\mathbf{93\%})$$

Which one do you prefer?

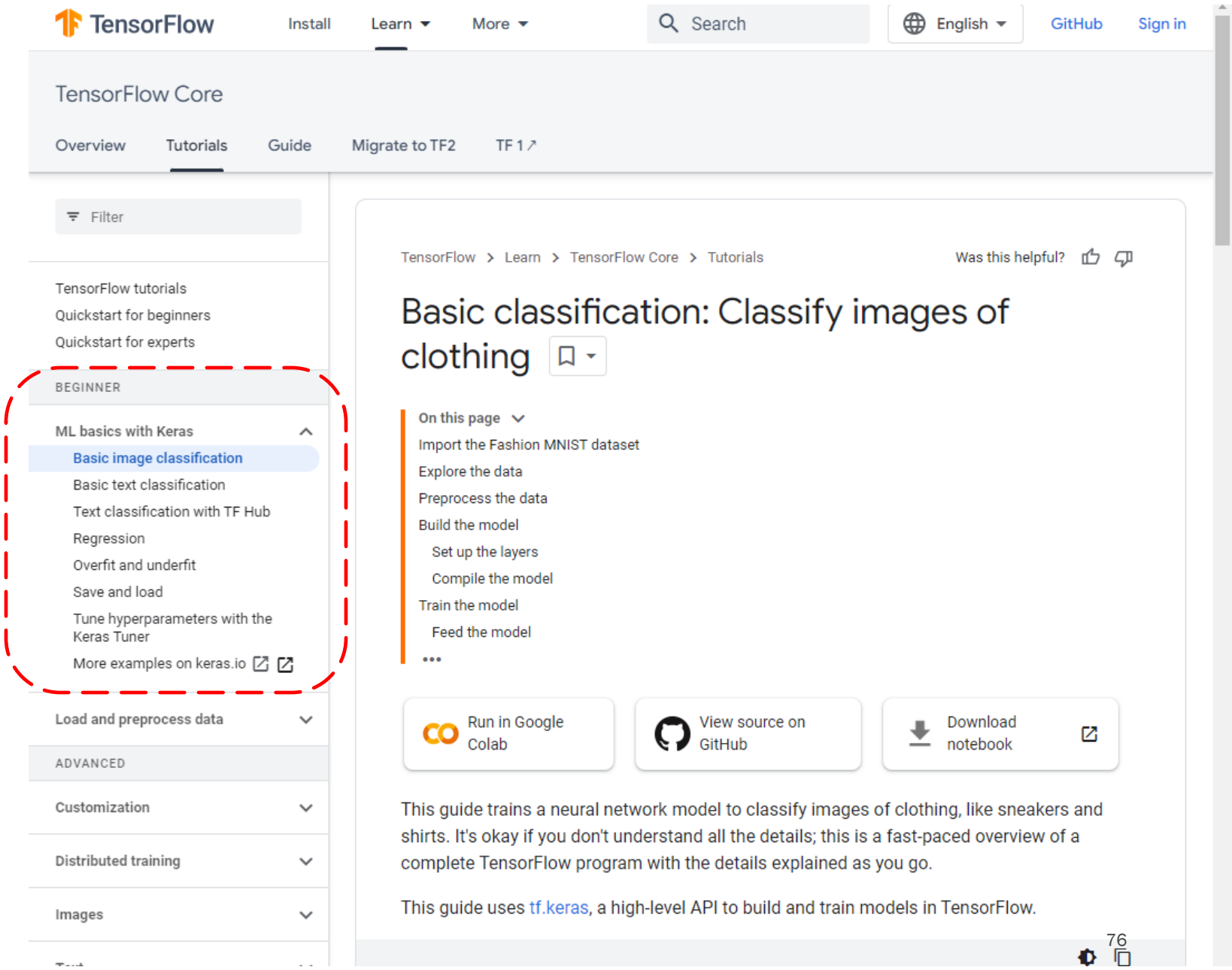
Final year project: Start with the question, not the dataset



Does the dataset have the answer to your question?

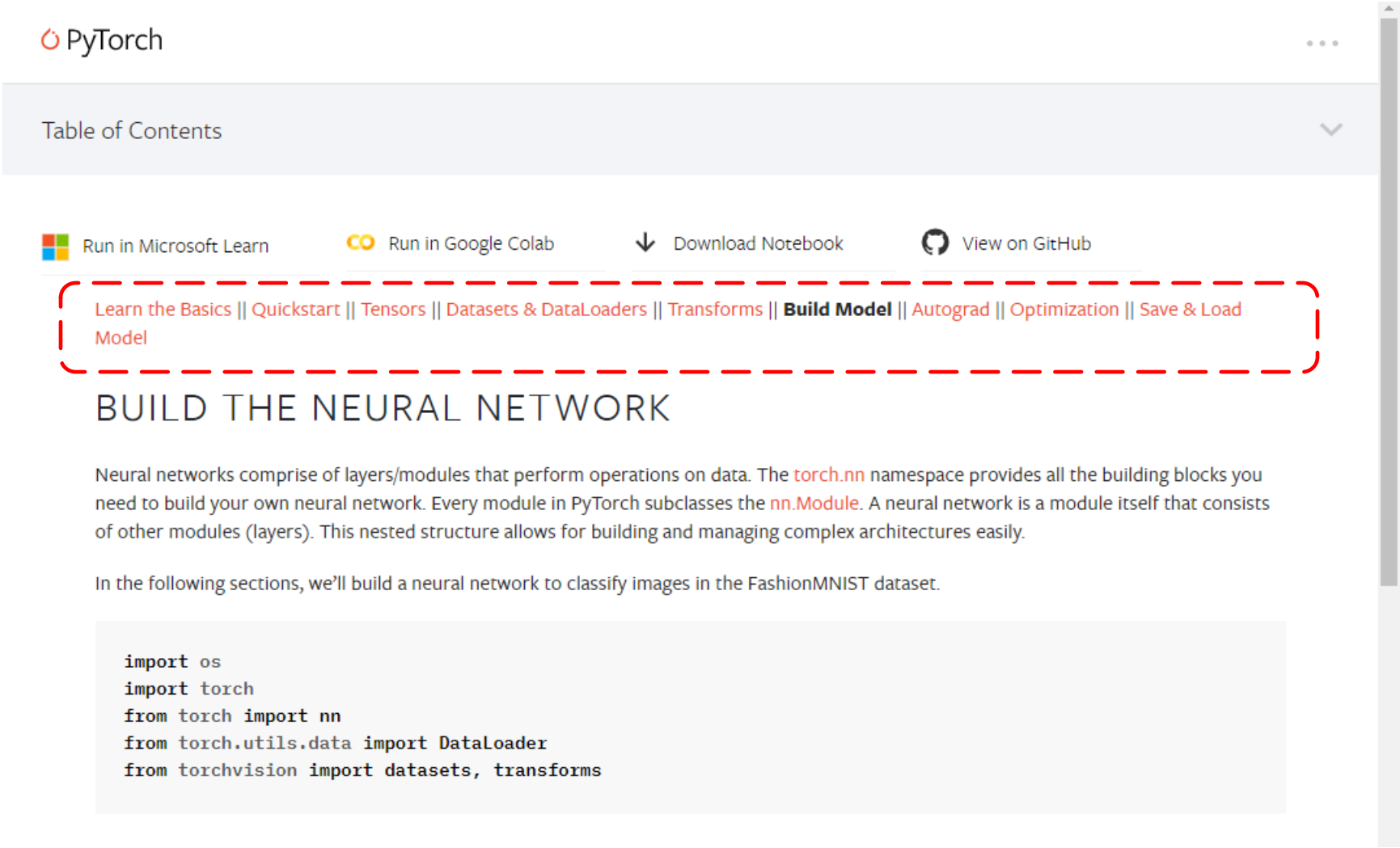
Tensorflow & KERAS

- https://www.tensorflow.org/tutorials/keras/classification



PyTorch

- https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html



THANK YOU!